

Áreas EDR

Table of contents

1	Áreas EDR	13
1.1	Áreas	13
1.2	Búsqueda y tags	14
1.3	Reglas de clasificación	14
1.4	Área 00 — General	14
1.5	Área 10 — Funcional	15
1.6	Área 20 — Arquitectura	15
1.7	Área 30 — Constraints	16
1.8	Área 40 — Datos	16
1.9	Área 50 — Integración	17
1.10	Área 60 — Seguridad	17
1.11	Área 70 — Calidad	18
1.12	Área 80 — Operación	18
1.13	Área 99 — Pendiente de clasificar	19
1.14	Criterio de asignación	19
1.15	Una única clasificación principal	19
1.16	El área no depende de la implementación	20
1.17	Estabilidad	20
1.18	Separación entre áreas	21
1.19	Creación de nuevas áreas	21
1.20	Principio de evolución	22
1.21	Regla fundamental	22
I	General	23
2	Registro único de decisiones de ingeniería	25
2.1	Contexto	25
2.2	Decisión	25
2.3	Alternativas consideradas	25
2.4	Consecuencias	25
3	Identificación y clasificación de los EDR	26
3.1	Contexto	26
3.2	Decisión	26

3.3	Alternativas consideradas	26
3.4	Consecuencias	26
4	Partir de la necesidad y no de las herramientas disponibles	27
4.1	Contexto	27
4.2	Decisión	27
4.3	Alternativas consideradas	27
4.4	Consecuencias	27
5	Encapsular la complejidad interna	28
5.1	Contexto	28
5.2	Decisión	28
5.3	Alternativas consideradas	28
5.4	Consecuencias	28
6	Automatizar lo repetible y mantener manual lo excepcional	29
6.1	Contexto	29
6.2	Decisión	29
6.3	Alternativas consideradas	29
6.4	Consecuencias	29
7	Limitar la velocidad del sistema a la capacidad humana sostenible	30
7.1	Contexto	30
7.2	Decisión	30
7.3	Alternativas consideradas	30
7.4	Consecuencias	30
8	Organizar la obra de IASI en tres volúmenes	31
8.1	Contexto	31
8.2	Decisión	31
8.3	Alternativas consideradas	31
8.4	Consecuencias	31
9	Separar el proceso vivido del resultado consolidado	32
9.1	Contexto	32
9.2	Decisión	32
9.3	Alternativas consideradas	32
9.4	Consecuencias	32
10	Hacer emerger las prácticas desde necesidades reales	33
10.1	Contexto	33
10.2	Decisión	33
10.3	Alternativas consideradas	33
10.4	Consecuencias	33

11 Adoptar desarrollo dirigido por especificaciones	34
11.1 Contexto	34
11.2 Decisión	34
11.3 Alternativas consideradas	34
11.4 Consecuencias	34
12 Lab execution model	35
13 Lab execution model	36
13.1 Contexto	36
13.2 Decisión	36
13.3 El lab como secuencia	37
13.4 Separación entre lab y proyecto	38
13.4.1 El lab contiene	38
13.4.2 El repositorio contiene	38
13.5 Estados oficiales del lab	38
13.6 Identidad y orden editorial	39
13.7 Historia del usuario	40
13.8 Regreso a un checkpoint	40
13.9 Reproducibilidad	41
13.10 Consecuencias	42
13.11 Principios derivados	42
14 Organizar el flujo del repositorio mediante nombres semánticos	43
15 Organizar el flujo del repositorio mediante nombres semánticos	44
15.1 Contexto	44
15.2 Decisión	44
15.3 Rationale	45
15.4 Alternativas consideradas	46
15.4.1 Mantener los prefijos A-, B-, C-, D-	46
15.4.2 Mantener <code>engine/</code>	46
15.4.3 Utilizar <code>model/</code>	46
15.4.4 Utilizar <code>deliverables/</code>	46
15.5 Consecuencias	46
15.6 Principios derivados	47
II Funcional	48
16 Usar iasi-home como puerta de entrada visual al ecosistema	50
16.1 Contexto	50
16.2 Decisión	50
16.3 Alternativas consideradas	50

16.4	Consecuencias	50
17	Distinguir libros de productos y artefactos en iasi-home	51
17.1	Contexto	51
17.2	Decisión	51
17.3	Alternativas consideradas	51
17.4	Consecuencias	51
18	Ofrecer iasi-home en castellano e inglés con detección automática	52
18.1	Contexto	52
18.2	Decisión	52
18.3	Alternativas consideradas	52
18.4	Consecuencias	52
19	Generar un catálogo por área de EDR	53
19.1	Contexto	53
19.2	Decisión	53
19.3	Alternativas consideradas	53
19.4	Consecuencias	53
20	Crear EDR mediante un Addin de RStudio	54
20.1	Contexto	54
20.2	Decisión	54
20.3	Alternativas consideradas	54
20.4	Consecuencias	54
21	Crear una presentación de introducción a IASI	55
21.1	Contexto	55
21.2	Decisión	55
21.3	Alternativas consideradas	55
21.4	Consecuencias	56
22	Proporcionar un agente IASI capaz de ejecutar trabajo sobre el workspace	57
22.1	Contexto	57
22.2	Decisión	57
22.3	Alternativas consideradas	58
22.3.1	Utilizar únicamente Ollama u Open WebUI	58
22.3.2	Depender exclusivamente de agentes externos	59
22.3.3	Construir inmediatamente un agente completo desde cero	59
22.4	Consecuencias	59

III Arquitectura	60
23 Mantener iasi-edr como repositorio independiente	62
23.1 Contexto	62
23.2 Decisión	62
23.3 Alternativas consideradas	62
23.4 Consecuencias	62
24 Publicar iasi-edr como Quarto book estructurado	63
24.1 Contexto	63
24.2 Decisión	63
24.3 Alternativas consideradas	63
24.4 Consecuencias	63
25 Definir las áreas EDR en _iasi.yml	64
25.1 Contexto	64
25.2 Decisión	64
25.3 Alternativas consideradas	64
25.4 Consecuencias	64
26 Organizar físicamente los EDR por área	65
26.1 Contexto	65
26.2 Decisión	65
26.3 Alternativas consideradas	65
26.4 Consecuencias	65
27 Regenerar los índices EDR en pre-render	66
27.1 Contexto	66
27.2 Decisión	66
27.3 Alternativas consideradas	66
27.4 Consecuencias	66
28 Mantener la lógica específica de EDR fuera de iasi.quarto	67
28.1 Contexto	67
28.2 Decisión	67
28.3 Alternativas consideradas	67
28.4 Consecuencias	67
29 Separar la configuración común de los perfiles HTML y PDF	68
29.1 Contexto	68
29.2 Decisión	68
29.3 Alternativas consideradas	68
29.4 Consecuencias	68

30 Usar la navbar como navegación principal de iasi-edr en HTML	69
30.1 Contexto	69
30.2 Decisión	69
30.3 Alternativas consideradas	69
30.4 Consecuencias	69
31 Componer el estilo HTML con SCSS común y específico	70
31.1 Contexto	70
31.2 Decisión	70
31.3 Alternativas consideradas	70
31.4 Consecuencias	70
32 Mantener iasi-graphics deliberadamente simple	71
32.1 Contexto	71
32.2 Decisión	71
32.3 Alternativas consideradas	71
32.4 Consecuencias	71
33 Crear un espacio dedicado para capacidades agentic de IASI	72
33.1 Contexto	72
33.2 Decisión	72
33.3 Alternativas consideradas	72
33.4 Consecuencias	72
34 Separar el entorno de laboratorios mediante una máquina virtual dedicada	73
34.1 Contexto	73
34.2 Decisión	73
34.3 Alternativas consideradas	73
34.4 Consecuencias	74
35 Usar HTMLSlides para materializar Introducing IASI	75
35.1 Contexto	75
35.2 Decisión	76
35.3 Alternativas consideradas	76
35.3.1 PowerPoint	76
35.3.2 Gamma	77
35.3.3 Prezi	77
35.3.4 HTML desarrollado específicamente para la presentación	78
35.3.5 HTMLSlides	78
35.4 Consecuencias	79
36 Mantener iasi-agent como proyecto separado de IASI	80
36.1 Contexto	80
36.2 Decisión	80

36.3	Alternativas consideradas	81
36.3.1	Incorporar el agente directamente dentro de IASI	81
36.3.2	Tratar el agente como parte de <code>iasi-dev</code>	81
36.3.3	Utilizar únicamente adaptadores para agentes externos	81
36.4	Consecuencias	81
37	Usar OpenCode como base inicial de <code>iasi-agent</code>	83
37.1	Contexto	83
37.2	Decisión	83
37.3	Alternativas consideradas	84
37.3.1	Ollama + Open WebUI	84
37.3.2	Continue	85
37.3.3	Aider	85
37.3.4	Codex CLI	86
37.3.5	Construir un agente desde cero	86
37.3.6	OpenCode	87
37.4	Consecuencias	88
IV	Constraints	89
38	Diseñar las publicaciones IASI para escritorio	91
38.1	Contexto	91
38.2	Decisión	91
38.3	Alternativas consideradas	91
38.4	Consecuencias	91
39	Evitar la generalización prematura	92
39.1	Contexto	92
39.2	Decisión	92
39.3	Alternativas consideradas	92
39.4	Consecuencias	92
40	Construir manualmente la infraestructura antes de automatizarla	93
40.1	Contexto	93
40.2	Decisión	93
40.3	Alternativas consideradas	93
40.4	Consecuencias	93
41	Usar Windows como sistema operativo de referencia para los labs	94
41.1	Contexto	94
41.2	Decisión	94
41.3	Alternativas consideradas	94
41.4	Consecuencias	95

V	Datos	96
42	Usar el front matter como fuente de metadatos del EDR	98
42.1	Contexto	98
42.2	Decisión	98
42.3	Alternativas consideradas	98
42.4	Consecuencias	98
43	Tratar los índices de área como datos derivados	99
43.1	Contexto	99
43.2	Decisión	99
43.3	Alternativas consideradas	99
43.4	Consecuencias	99
44	Hacer los EDR consultables por metadatos y texto completo	100
44.1	Contexto	100
44.2	Decisión	100
44.3	Alternativas consideradas	100
44.4	Consecuencias	100
VI	Integración	101
45	Distinguir MCP del servidor MCP	103
45.1	Contexto	103
45.2	Decisión	103
45.3	Alternativas consideradas	103
45.4	Consecuencias	103
46	Exponer PlantUML como capacidad utilizable por modelos	104
46.1	Contexto	104
46.2	Decisión	104
46.3	Alternativas consideradas	104
46.4	Consecuencias	104
47	Clasificar los productos técnicos por su forma de uso	105
47.1	Contexto	105
47.2	Decisión	105
47.3	Alternativas consideradas	105
47.4	Consecuencias	105
VII	Seguridad	106

VIII Calidad	108
48 Validar la estructura Cuarto antes de preparar o renderizar	110
48.1 Contexto	110
48.2 Decisión	110
48.3 Alternativas consideradas	110
48.4 Consecuencias	110
49 Fallar ante inconsistencias estructurales del registro EDR	111
49.1 Contexto	111
49.2 Decisión	111
49.3 Alternativas consideradas	111
49.4 Consecuencias	111
50 Regenerar de forma determinista los artefactos derivados	112
50.1 Contexto	112
50.2 Decisión	112
50.3 Alternativas consideradas	112
50.4 Consecuencias	112
51 Preservar el formato deliberado del código Go	113
51.1 Contexto	113
51.2 Decisión	113
51.3 Rationale	114
51.4 Alternativas consideradas	114
51.4.1 Mantener <code>gofmt</code> como obligatorio	114
51.4.2 Utilizar <code>gofmt</code> por defecto y permitir excepciones puntuales	114
51.4.3 Desactivar únicamente el formateo al guardar	114
51.5 Consecuencias	115
51.6 Principios derivados	115
IX Operación	116
52 Usar HTML como perfil por defecto	118
52.1 Contexto	118
52.2 Decisión	118
52.3 Alternativas consideradas	118
52.4 Consecuencias	118
53 Separar los directorios de salida por perfil	119
53.1 Contexto	119
53.2 Decisión	119
53.3 Alternativas consideradas	119

53.4	Consecuencias	119
54	Usar scripts R reutilizables para preview y render	120
54.1	Contexto	120
54.2	Decisión	120
54.3	Alternativas consideradas	120
54.4	Consecuencias	120
55	Separar herramientas principales de trabajo y laboratorios locales	121
55.1	Contexto	121
55.2	Decisión	121
55.3	Alternativas consideradas	121
55.4	Consecuencias	121
56	Postprocesar la salida de Quarto sin forzar su comportamiento	122
56.1	Contexto	122
56.2	Decisión	122
56.3	Alternativas consideradas	123
56.4	Consecuencias	123
X	Pendiente de clasificar	124
57	Áreas EDR	126
58	Áreas EDR	127
58.1	Áreas	127
58.2	Área 10 — Funcional	128
58.3	Área 20 — Arquitectura	129
58.4	Área 30 — Constraints	129
58.5	Área 40 — Datos	130
58.6	Área 50 — Integración	130
58.7	Área 60 — Seguridad	131
58.8	Área 70 — Calidad	131
58.9	Área 80 — Operación	132
58.10	Área 99 — Pendiente de clasificar	132
58.11	Criterio de asignación	133
58.12	Una única clasificación principal	133
58.13	El área no depende de la implementación	134
58.14	Estabilidad	134
58.15	Separación entre áreas	134
58.16	Creación de nuevas áreas	135
58.17	Principio de evolución	135

58.18Regla fundamental	136
----------------------------------	-----

1 Áreas EDR

Los Engineering Decision Records de IASI utilizan identificadores con la forma:

EDR-**nnmmm**

donde:

- **nn** identifica el **área principal de ingeniería** de la decisión.
- **mmm** identifica secuencialmente la decisión dentro de esa área.

El área permite conocer la naturaleza principal de una decisión antes incluso de abrir el EDR.

1.1 Áreas

nn	Área	Criterio
00	General	Decisiones transversales o relativas al propio funcionamiento de IASI.
10	Funcional	Qué debe hacer el sistema y qué comportamiento debe ofrecer.
20	Arquitectura	Estructura del sistema, componentes, responsabilidades y relaciones.
30	Constraints	Restricciones que condicionan o limitan las posibles soluciones.
40	Datos	Modelos, persistencia, intercambio, ciclo de vida y gobierno del dato.
50	Integración	Interfaces, protocolos y comunicación entre sistemas o componentes.
60	Seguridad	Identidad, acceso, protección, amenazas y controles.
70	Calidad	Validación, pruebas, observabilidad y atributos de calidad.
80	Operación	Despliegue, ejecución, configuración, infraestructura y explotación.
99	Pendiente de clasificar	Decisiones registradas provisionalmente cuya clasificación definitiva todavía no se ha determinado.

1.2 Búsqueda y tags

En la publicación HTML, la lupa realiza una búsqueda de texto completo sobre el libro.

Cada EDR pertenece a una única **área principal**, pero puede incorporar varios **tags** para describir temas transversales. Los tags se definen una sola vez en el front matter y se muestran automáticamente en la página del EDR, por lo que también forman parte de la búsqueda.

La regla es sencilla:

- el **área** responde a dónde se clasifica principalmente una decisión;
- los **tags** responden a de qué habla la decisión y permiten encontrarla desde otros ángulos.

1.3 Reglas de clasificación

Cada EDR pertenece a **una única área principal**.

El área representa la naturaleza de la decisión, no:

- la herramienta utilizada;
- la tecnología elegida;
- el repositorio donde se implementa;
- el equipo responsable.

Cuando una decisión afecta a varios ámbitos, se asigna al área que mejor represente **el objeto principal de la decisión**.

Los efectos secundarios pueden expresarse mediante los metadatos del EDR.

1.4 Área 00 — General

00 está reservado para decisiones generales o transversales.

También pertenece a esta área cualquier decisión relacionada con el propio sistema de Engineering Decision Records.

Por ejemplo:

EDR-00001

puede definir el modelo, estructura y reglas de los propios EDR.

00 no debe utilizarse simplemente porque una decisión resulte difícil de clasificar.

1.5 Área 10 — Funcional

El área 10 agrupa decisiones relacionadas con **qué debe hacer el sistema**.

Incluye decisiones sobre:

- comportamiento funcional;
- capacidades ofrecidas;
- reglas de negocio;
- interacción entre el sistema y sus usuarios;
- resultados esperados;
- criterios funcionales.

Una decisión pertenece a esta área cuando su cuestión principal puede formularse aproximadamente como:

¿Qué comportamiento debe ofrecer el sistema?

1.6 Área 20 — Arquitectura

El área 20 agrupa decisiones relacionadas con **cómo se estructura el sistema**.

Incluye decisiones sobre:

- componentes;
- responsabilidades;
- relaciones entre componentes;
- separación de responsabilidades;
- organización interna;
- estilos y patrones arquitectónicos;
- distribución de capacidades;
- fronteras entre subsistemas.

Una decisión pertenece a esta área cuando su cuestión principal puede formularse aproximadamente como:

¿Cómo organizamos el sistema para satisfacer sus necesidades?

1.7 Área 30 — Constraints

El área 30 agrupa decisiones derivadas de **restricciones que condicionan el espacio de soluciones posibles**.

Incluye restricciones:

- técnicas;
- organizativas;
- operativas;
- regulatorias;
- económicas;
- temporales;
- de compatibilidad;
- de plataforma;
- impuestas por sistemas externos.

Una decisión pertenece a esta área cuando su cuestión principal parte de una condición que no puede ignorarse libremente.

Los constraints no describen necesariamente qué queremos hacer, sino **qué límites debe respetar cualquier solución válida**.

1.8 Área 40 — Datos

El área 40 agrupa decisiones relacionadas con la **representación, persistencia, intercambio y ciclo de vida de los datos**.

Incluye decisiones sobre:

- modelos de datos;
- estructuras y formatos;
- almacenamiento y persistencia;
- migraciones;
- retención y ciclo de vida;
- calidad y gobierno del dato;
- intercambio y transformación de información.

Una decisión pertenece a esta área cuando el elemento principal sobre el que se decide es el dato y su tratamiento.

1.9 Área 50 — Integración

El área 50 agrupa decisiones relacionadas con la **comunicación entre sistemas, servicios o componentes**.

Incluye decisiones sobre:

- interfaces;
- APIs;
- protocolos;
- mensajería;
- contratos de integración;
- interoperabilidad;
- mecanismos de comunicación;
- dependencias con sistemas externos.

Una decisión pertenece a esta área cuando su cuestión principal es cómo se relacionan o intercambian información distintos elementos del sistema.

1.10 Área 60 — Seguridad

El área 60 agrupa decisiones relacionadas con la **protección del sistema, sus usuarios y su información**.

Incluye decisiones sobre:

- identidad;
- autenticación;
- autorización;
- gestión de secretos;
- protección de datos;
- amenazas;
- controles;
- aislamiento;
- trazabilidad de seguridad.

Una decisión pertenece a esta área cuando su propósito principal es reducir riesgos o establecer mecanismos de protección.

1.11 Área 70 — Calidad

El área 70 agrupa decisiones relacionadas con la **validación del sistema y sus atributos de calidad**.

Incluye decisiones sobre:

- pruebas;
- acceptance tests;
- validación;
- verificación;
- observabilidad;
- métricas;
- rendimiento;
- fiabilidad;
- mantenibilidad;
- criterios de calidad.

Una decisión pertenece a esta área cuando define cómo demostrar, medir o asegurar que el sistema cumple las propiedades esperadas.

1.12 Área 80 — Operación

El área 80 agrupa decisiones relacionadas con la **ejecución y explotación del sistema**.

Incluye decisiones sobre:

- despliegue;
- configuración;
- infraestructura;
- entornos;
- ejecución;
- automatización operativa;
- monitorización;
- recuperación;
- mantenimiento en producción.

Una decisión pertenece a esta área cuando su cuestión principal es cómo ejecutar, desplegar, configurar o mantener el sistema en funcionamiento.

1.13 Área 99 — Pendiente de clasificar

99 está reservado para decisiones que deben registrarse, pero cuya clasificación definitiva todavía no está clara.

Su uso es **provisional**.

Debe utilizarse para evitar dos problemas:

- retrasar el registro de una decisión porque todavía no se sabe dónde clasificarla;
- forzar una clasificación prematura que después resulte incorrecta.

99 no es una categoría residual ni un destino permanente.

Cuando se determine el área correcta, la decisión debe trasladarse a su clasificación definitiva.

1.14 Criterio de asignación

Para determinar **nn**, debe identificarse primero la naturaleza principal de la decisión.

Puede utilizarse esta secuencia:

1. ¿Cuál es el problema que estamos resolviendo?
2. ¿Qué aspecto de la ingeniería estamos decidiendo?
3. ¿Dónde tendría sentido buscar esta decisión dentro de varios años?
4. ¿Seguiría perteneciendo a la misma área si cambiase la tecnología utilizada?

La respuesta determina el área principal.

En caso de duda entre varias áreas, debe seleccionarse aquella que mejor represente **la razón por la que existe la decisión**.

Si todavía no existe información suficiente para hacerlo con criterio, se utiliza provisionalmente el área 99.

1.15 Una única clasificación principal

Una decisión puede afectar simultáneamente a múltiples ámbitos.

Por ejemplo, una decisión arquitectónica puede tener consecuencias funcionales, operativas o de seguridad.

Esto no implica que deba pertenecer a varias áreas.

Cada EDR tiene un único **nn**.

Los demás efectos se expresan mediante metadatos como:

```
tags:
  - publication
  - validation

affects:
  - iasi-quarto
  - iasi-book-I

related:
  - EDR-20002
```

El identificador proporciona la clasificación principal.

Los metadatos proporcionan el contexto adicional.

1.16 El área no depende de la implementación

nn no debe asignarse según:

- el lenguaje de programación;
- el framework;
- la herramienta;
- el producto;
- el repositorio;
- el proveedor;
- el agente de IA que implemente la decisión.

Por ejemplo, una decisión sobre cómo se estructura la publicación de IASI puede implementarse inicialmente en `iasi-quarto`.

Eso no convierte automáticamente la decisión en una decisión sobre Quarto.

La pregunta relevante es qué se está decidiendo desde el punto de vista de ingeniería.

1.17 Estabilidad

Una vez asignado un código nn a un área:

- su significado no cambia;
- no se reutiliza para otro ámbito;

- los EDR clasificados conservan su identificador.

Los códigos de área son permanentes.

El área 99 constituye la excepción: representa explícitamente un estado provisional pendiente de clasificación.

1.18 Separación entre áreas

Los códigos se asignan con separación deliberada:

```
00
10
20
30
40
50
60
70
80
99
```

Esto mantiene una clasificación legible y deja espacio para incorporar nuevas áreas si la evolución de IASI lo hace necesario.

1.19 Creación de nuevas áreas

La lista de áreas no pretende definir anticipadamente todos los ámbitos posibles de la ingeniería.

Se crearán nuevas áreas cuando las decisiones reales de IASI hagan necesaria una nueva categoría estable.

Antes de crear una nueva área debe comprobarse que:

1. ninguna de las áreas existentes representa adecuadamente la decisión;
2. el nuevo ámbito tiene significado propio desde el punto de vista de ingeniería;
3. no representa simplemente una herramienta, tecnología o producto;
4. previsiblemente podrá contener múltiples decisiones;
5. su significado puede mantenerse estable en el tiempo.

La taxonomía debe evolucionar a partir de necesidades reales.

1.20 Principio de evolución

IASI no pretende construir anticipadamente una clasificación completa de todas las decisiones de ingeniería posibles.

Las áreas se incorporarán cuando aparezcan decisiones reales que justifiquen su existencia.

Las áreas no se inventan por anticipado. Se descubren cuando las decisiones de ingeniería las hacen necesarias.

1.21 Regla fundamental

nn clasifica la naturaleza principal de la decisión. Los metadatos describen todo lo demás.

El identificador debe permitir saber dónde buscar una decisión.

El contenido y los metadatos explican su alcance, sus relaciones y sus consecuencias.

Part I

General

Esta sección reúne los Engineering Decision Records clasificados en el área **General** (00).

ID	Título
EDR-00001	Registro único de decisiones de ingeniería
EDR-00002	Identificación y clasificación de los EDR
EDR-00003	Partir de la necesidad y no de las herramientas disponibles
EDR-00004	Encapsular la complejidad interna
EDR-00005	Automatizar lo repetible y mantener manual lo excepcional
EDR-00006	Limitar la velocidad del sistema a la capacidad humana sostenible
EDR-00007	Organizar la obra de IASI en tres volúmenes
EDR-00008	Separar el proceso vivido del resultado consolidado
EDR-00009	Hacer emerger las prácticas desde necesidades reales
EDR-00010	Adoptar desarrollo dirigido por especificaciones
EDR-00011	Lab execution model
EDR-00012	Organizar el flujo del repositorio mediante nombres semánticos

2 Registro único de decisiones de ingeniería

2.1 Contexto

Las decisiones de IASI aparecen durante el desarrollo de libros, herramientas, infraestructura, experimentos y conversaciones de diseño. Si cada producto conserva su propia versión de una decisión, el razonamiento termina duplicado y puede divergir.

2.2 Decisión

`iasi-edr` será el registro único de las decisiones de ingeniería de IASI. Los libros, artefactos, productos y el Journey pueden explicar o referenciar una decisión, pero no mantienen copias independientes de su contenido normativo.

2.3 Alternativas consideradas

- Mantener ADR o notas de decisión dentro de cada repositorio.
- Documentar únicamente el estado final en los libros.
- Conservar las decisiones solo en el Journey.

2.4 Consecuencias

Existe un punto único para consultar por qué se tomó una decisión. Se reduce la duplicación y se hace posible referenciar una identidad estable desde cualquier pieza del ecosistema.

3 Identificación y clasificación de los EDR

3.1 Contexto

Un registro central necesita una identidad estable y una clasificación suficientemente amplia para localizar decisiones funcionales, arquitectónicas, operativas, de datos, calidad e integración sin reducir EDR a decisiones puramente arquitectónicas.

3.2 Decisión

Los documentos usarán el identificador EDR-**nn**mmm, donde **nn** representa el área principal y **mmm** la secuencia dentro de esa área. 00 queda reservado para decisiones generales o transversales. Las áreas son: 00 General, 10 Funcional, 20 Arquitectura, 30 Constraints, 40 Datos, 50 Integración, 60 Seguridad, 70 Calidad, 80 Operación y 99 Pendiente de clasificar. Los identificadores definitivos son permanentes y no se reutilizan.

3.3 Alternativas consideradas

- Una secuencia global sin clasificación.
- Identificadores derivados únicamente del nombre del archivo.
- Numeración por repositorio o producto.

3.4 Consecuencias

El propio identificador aporta una primera señal de clasificación, permite ordenar documentos y mantiene referencias estables a lo largo del ecosistema.

4 Partir de la necesidad y no de las herramientas disponibles

4.1 Contexto

Empezar preguntando cómo resolver un problema con las herramientas actuales condiciona el espacio de solución antes de haber entendido la necesidad. La capacidad disponible termina dictando el diseño.

4.2 Decisión

Ante un problema se preguntará primero qué se necesita para resolverlo. Solo después se evaluará si herramientas, tecnologías o prácticas existentes satisfacen esa necesidad. Si no lo hacen, se adaptarán o se crearán los mecanismos necesarios.

4.3 Alternativas consideradas

- Elegir primero una herramienta y adaptar el problema a sus capacidades.
- Limitar la solución al catálogo tecnológico ya disponible.

4.4 Consecuencias

Las decisiones se justifican por la necesidad que resuelven y no por familiaridad tecnológica. La metodología mantiene abierto el espacio de diseño y obliga a cuestionar incluso las soluciones aparentemente establecidas.

5 Encapsular la complejidad interna

5.1 Contexto

Una solución puede requerir mecanismos internos complejos sin que esa complejidad deba trasladarse a quien la utiliza. Exponer cada detalle técnico convierte la potencia interna en fricción operativa.

5.2 Decisión

Los productos IASI deben encapsular la complejidad técnica detrás de interfaces, comandos y contratos simples. La complejidad se admite donde sea necesaria internamente, pero no se considera aceptable como coste impuesto al usuario.

5.3 Alternativas consideradas

- Exponer directamente todos los componentes internos.
- Simplificar eliminando capacidades necesarias.

5.4 Consecuencias

El diseño de interfaces pasa a ser parte de la ingeniería, no una capa cosmética. Una implementación compleja puede seguir ofreciendo un uso pequeño, explícito y predecible.

6 Automatizar lo repetible y mantener manual lo excepcional

6.1 Contexto

No toda tarea merece convertirse en un sistema general. Algunas aparecen con frecuencia y tienen una estructura clara; otras son excepcionales, visuales o requieren iteración específica.

6.2 Decisión

IASI automatizará los casos repetibles, estructurables y suficientemente frecuentes. Los casos excepcionales permanecerán manuales o se resolverán con herramientas especializadas cuando aparezcan.

6.3 Alternativas consideradas

- Automatizar desde el principio cualquier caso imaginable.
- Renunciar a automatizar y resolver todo manualmente.

6.4 Consecuencias

Se reduce la superficie de mantenimiento y la complejidad accidental. La automatización crece cuando existe evidencia de repetición, no como anticipación especulativa.

7 Limitar la velocidad del sistema a la capacidad humana sostenible

7.1 Contexto

La disponibilidad continua de agentes elimina esperas que antes actuaban como pausas naturales. El humano puede encadenar decisiones de alta intensidad mientras la IA resuelve tareas paralelas, incluso cuando su capacidad de comprender y validar ya está disminuyendo.

7.2 Decisión

La velocidad sostenible de IASI se diseñará alrededor de la capacidad humana para comprender, decidir, validar y asimilar, no alrededor de la velocidad máxima de los agentes. No se confundirá disponibilidad de la IA con disponibilidad humana.

7.3 Alternativas consideradas

- Maximizar siempre el paralelismo de agentes.
- Medir productividad solo por volumen o tiempo de ejecución.

7.4 Consecuencias

Los workflows deben admitir pausas, límites y momentos de asimilación. El rendimiento se evalúa por la calidad sostenible de las decisiones, no por mantener ocupados todos los agentes.

8 Organizar la obra de IASI en tres volúmenes

8.1 Contexto

El material de IASI mezcla fundamentos, historia real de construcción y metodología consolidada. Un único volumen obliga a mezclar el porqué, el proceso y el resultado final.

8.2 Decisión

La obra se separará en tres volúmenes: Volumen I para fundamentos y teoría, Volumen II para el viaje, errores y evolución, y Volumen III para la metodología o framework consolidado. Las partes se presentan con numeración romana y los capítulos mantienen numeración arábica.

8.3 Alternativas consideradas

- Un único libro cronológico.
- Un libro puramente metodológico que oculte el proceso real.
- Repartir el contenido por tecnologías en lugar de por función narrativa.

8.4 Consecuencias

Cada volumen puede mantener una voz y propósito propios sin perder coherencia. El lector puede distinguir teoría, evidencia del proceso y método final.

9 Separar el proceso vivido del resultado consolidado

9.1 Contexto

Las metodologías suelen describir únicamente el resultado final, pero IASI también quiere conservar el proceso efectivo: discusiones, errores, cambios de criterio y descubrimientos que llevaron a ese estado.

9.2 Decisión

El Book contiene la versión final y correcta del conocimiento. *iasi-journey* conserva cómo se llegó a ella. Las entradas del Journey se tratan como registros inmutables del proceso y no se reescriben para hacer que el pasado parezca coherente con la decisión final.

9.3 Alternativas consideradas

- Reescribir el Journey cuando cambia la conclusión.
- Incluir toda la conversación histórica dentro del Book.

9.4 Consecuencias

IASI puede mostrar tanto el conocimiento consolidado como el proceso real que lo produjo. La discrepancia histórica deja de ser ruido y se convierte en evidencia de evolución.

10 Hacer emerger las prácticas desde necesidades reales

10.1 Contexto

IASI es a la vez metodología y caso de estudio de su propia construcción. Introducir de antemano skills, especificaciones, agentes o procesos completos convertiría el proyecto en una demostración prefabricada.

10.2 Decisión

Las prácticas, skills, automatizaciones y mecanismos de IASI deben surgir cuando una necesidad real del propio ecosistema los justifique. IASI se utilizará como caso de estudio vivo para validar esas decisiones.

10.3 Alternativas consideradas

- Diseñar por adelantado el framework completo y después buscar ejemplos.
- Adoptar una práctica porque sea habitual en otros ecosistemas.

10.4 Consecuencias

Cada componente puede señalar la necesidad concreta que lo originó. El framework final se apoya en experiencia acumulada y no solo en diseño teórico.

11 Adoptar desarrollo dirigido por especificaciones

11.1 Contexto

La reducción del coste de implementación aumenta el valor relativo de decidir y especificar correctamente qué debe construirse. La implementación, humana o asistida por IA, no debe convertirse en el lugar donde se descubre implícitamente el comportamiento esperado.

11.2 Decisión

IASI adopta desarrollo dirigido por especificaciones. La especificación explícita es un artefacto principal y precede a la implementación cuando el cambio lo requiere. OpenSpec se utiliza como mecanismo para materializar este enfoque allí donde resulte aplicable.

11.3 Alternativas consideradas

- Implementar primero y documentar después.
- Confiar en prompts o conversaciones efímeras como única especificación.

11.4 Consecuencias

Las implementaciones pueden cambiar de herramienta o agente sin perder la intención. La revisión se centra primero en qué debe ocurrir y después en cómo se implementa.

12 Lab execution model

13 Lab execution model

13.1 Contexto

Algunos labs de IASI pueden resolverse como ejercicios pequeños y autocontenidos. Otros, sin embargo, deben trabajar sobre **proyectos reales**, con repositorios propios, historia Git, código, configuración y evolución independiente.

`iasi-graphics` es un ejemplo de este segundo caso.

El objetivo del lab no es simplemente construir el proyecto final. El objetivo es **observar y validar cómo IASI avanza a medida que recibe información**, qué puede hacer en cada momento, qué debe rechazar por falta de información y cómo transforma sucesivos inputs en decisiones, definitions, planes y resultados.

El lab debe permitir repetir ese proceso de forma controlada y reproducible.

13.2 Decisión

Los labs que trabajen sobre proyectos reales se ejecutarán sobre el **repositorio real del proyecto**.

El lab define el experimento y su secuencia de pasos. El repositorio contiene el estado real producido durante su ejecución.

Book II / Lab

define y explica

secuencia de pasos

inputs sucesivos

repositorio real

estado paso 1

estado paso 2

```
estado paso 3
...
```

No se mantendrá una copia paralela del proyecto dentro del lab.

13.3 El lab como secuencia

Un lab no representa únicamente un resultado final.

Representa una **secuencia controlada de estados**.

Por ejemplo, un lab sobre **iasi-graphics** podría comenzar con un input externo:

```
Queremos hacer un filtro y un MCP que genere gráficos.
```

Con esa información, IASI puede entender parte de la intención, pero no dispone necesariamente de información suficiente para diseñar o implementar la solución.

El comportamiento correcto puede ser detenerse y declarar que existen huecos que impiden continuar.

Posteriormente puede recibirse un nuevo input interno:

```
Por ahora únicamente crea el proyecto.
```

Esta nueva instrucción sí permite realizar una acción concreta: crear el proyecto, aunque siga sin existir información suficiente para diseñar el sistema completo.

El lab continúa de esta forma:

```
input
↓
análisis
↓
estado de conocimiento
↓
acción posible o bloqueo
↓
resultado
↓
nuevo input
↓
...
```

Cada paso valida que IASI actúa únicamente con la información disponible en ese momento.

IASI no debe anticipar decisiones futuras ni completar mediante suposiciones aquello que todavía no ha sido definido.

13.4 Separación entre lab y proyecto

El lab y el proyecto tienen responsabilidades diferentes.

13.4.1 El lab contiene

- la pregunta o propósito del experimento;
- la secuencia de pasos;
- los inputs que se introducen en cada paso;
- las condiciones esperadas;
- aquello que IASI puede hacer;
- aquello que IASI todavía no debe hacer;
- las evidencias y conclusiones del experimento.

13.4.2 El repositorio contiene

- el proyecto real;
- los inputs incorporados durante la ejecución;
- las definiciones producidas;
- los planes;
- el código;
- la configuración;
- los artefactos generados;
- la historia Git resultante.

Por tanto:

El lab controla el experimento; el repositorio contiene el proyecto.

13.5 Estados oficiales del lab

Cada paso significativo tendrá asociado un **estado oficial reproducible** del repositorio.

Estos estados se identificarán mediante tags Git.

Por ejemplo:

```
lab-graphics-step-01  
lab-graphics-step-02  
lab-graphics-step-03
```

Cada tag identifica el estado exacto del proyecto correspondiente a un determinado punto del experimento.

Los tags forman parte del repositorio oficial utilizado para ejecutar el lab.

No representan la historia particular de cada usuario.

Representan los **checkpoints oficiales del experimento**.

13.6 Identidad y orden editorial

El número utilizado para ordenar un lab dentro de Book II no forma parte de su identidad.

Por ejemplo:

```
005-graphics/
```

puede pasar posteriormente a:

```
010-graphics/
```

sin que la identidad del lab cambie.

Por esta razón, los tags no incorporarán el número editorial.

Se utilizará:

```
lab-graphics-step-01
```

y no:

```
lab-005-step-01
```

Esto evita que una reorganización editorial obligue a modificar la historia Git de los proyectos.

El principio es:

El número ordena. El nombre identifica.

13.7 Historia del usuario

Una vez iniciado el lab, el repositorio pertenece operativamente al usuario.

El usuario puede:

- modificar archivos;
- experimentar;
- cometer errores;
- crear sus propios commits;
- crear ramas;
- realizar soluciones alternativas;
- desviarse de la secuencia prevista.

IASI Labs no necesita controlar esa historia.

Los commits creados durante el trabajo son parte de la experiencia del usuario.

Los tags oficiales permanecen como referencias estables.

Por tanto:

Los commits son del usuario; los tags son del lab.

13.8 Regreso a un checkpoint

Si el usuario se pierde, rompe el proyecto o desea repetir una parte del experimento, puede regresar a cualquiera de los estados oficiales.

Por ejemplo:

```
git switch --detach lab-graphics-step-05
```

El repositorio volverá al estado exacto correspondiente a ese paso.

Desde ahí puede:

- inspeccionar el estado;
- repetir el paso;
- crear una nueva rama;
- continuar el experimento de otra forma.

El usuario puede haber realizado posteriormente otros labs o haber avanzado mucho más en el repositorio.

Eso no impide volver a un checkpoint anterior.

Por ejemplo, puede haber completado:

```
lab-graphics-step-05  
lab-mcp-step-03
```

y regresar conscientemente a:

```
lab-graphics-step-05
```

El estado posterior dejará de estar visible en ese checkout, pero no desaparece.

Puede recuperarse nuevamente mediante su tag correspondiente.

Los tags funcionan así como **puntos de restauración oficiales del recorrido experimental**.

13.9 Reproducibilidad

Este modelo permite que distintas personas ejecuten el mismo lab manteniendo historias Git completamente diferentes.

Dos usuarios pueden partir del mismo checkpoint:

```
lab-graphics-step-03
```

y producir soluciones diferentes.

El lab sigue siendo reproducible porque ambos conocen exactamente el estado inicial desde el que parte el paso.

La reproducibilidad no exige que todos produzcan los mismos commits.

Exige que los puntos de partida oficiales sean conocidos y recuperables.

13.10 Consecuencias

Este modelo permite que un lab evolucione naturalmente hacia un proyecto real sin necesidad de mantener una versión artificial específica para formación.

El mismo repositorio puede ser simultáneamente:

- un proyecto real de IASI;
- el sujeto de uno o varios labs;
- una fuente de evidencias sobre el funcionamiento de la metodología;
- un entorno donde el usuario puede experimentar libremente.

El lab no simula el desarrollo del proyecto.

El lab utiliza el desarrollo real como experimento.

Esto permite probar no solo el resultado final de IASI, sino su comportamiento durante todo el proceso: cuándo puede avanzar, cuándo debe detenerse, qué información utiliza y cómo cambia el proyecto conforme aparecen nuevos inputs.

13.11 Principios derivados

El lab controla el experimento; el repositorio contiene el proyecto.

Los commits son del usuario; los tags son del lab.

El número ordena; el nombre identifica.

Cada paso del lab debe poder partir de un estado oficial conocido.

IASI debe actuar únicamente con la información disponible en cada paso.

Los labs no simulan proyectos reales: pueden ejecutarse directamente sobre ellos.

14 Organizar el flujo del repositorio mediante nombres semánticos

15 Organizar el flujo del repositorio mediante nombres semánticos

15.1 Contexto

Durante la definición inicial del flujo de trabajo se utilizaron prefijos alfabéticos para ordenar visualmente las áreas principales del repositorio:

```
A-inputs/  
B-engine/  
C-outputs/  
D-deliverables/
```

Esta estructura hacía visible una secuencia aproximada de trabajo, pero introducía dos problemas.

En primer lugar, los prefijos A-, B-, C- y D- no aportan significado propio. Su función es únicamente forzar el orden de las carpetas.

En segundo lugar, ese orden puede sugerir un proceso estrictamente secuencial por fases, cuando el trabajo real puede volver sobre los inputs, revisar especificaciones, regenerar outputs o repetir validaciones tantas veces como sea necesario.

Además, algunos nombres resultaban demasiado ambiguos. En particular, **engine/** podía interpretarse como un componente software ejecutable, mientras que su contenido representa realmente los elementos que coordinan y gobiernan el proceso de trabajo.

15.2 Decisión

Las áreas principales del repositorio se nombrarán por su **función**, sin prefijos artificiales destinados únicamente a imponer un orden visual.

La estructura canónica será:

```
inputs/  
orchestration/  
outputs/  
releases/
```

Cada área representa una responsabilidad diferente:

- **inputs/** contiene la información que entra en el proceso.
- **orchestration/** contiene los elementos que organizan, dirigen y gobiernan el trabajo realizado sobre los inputs, como especificaciones, tareas y otros mecanismos de coordinación.
- **outputs/** contiene los artefactos producidos durante el proceso.
- **releases/** contiene los artefactos aceptados y preparados para su publicación, entrega o distribución.

La estructura no representa fases rígidas ni obliga a recorrer las carpetas una sola vez en ese orden.

15.3 Rationale

Los nombres deben expresar **qué es cada cosa**, no únicamente dónde queremos que aparezca en un árbol de directorios.

Los prefijos de orden convierten una decisión de presentación en parte permanente del modelo del repositorio. Al eliminarlos, la estructura queda definida por conceptos que pueden evolucionar sin depender de una numeración o secuencia artificial.

orchestration/ sustituye a **engine/** porque describe mejor la responsabilidad de su contenido. No representa necesariamente un motor ejecutable, sino el conjunto de artefactos que coordinan el proceso, actualmente incluyendo elementos como:

```
orchestration/  
  specifications/  
  tasks/
```

y pudiendo incorporar en el futuro otros elementos de gobierno o coordinación sin alterar el significado del área.

releases/ sustituye a **deliverables/** para distinguir entre aquello que el proceso **produce** y aquello que finalmente se **acepta y libera**.

15.4 Alternativas consideradas

15.4.1 Mantener los prefijos A-, B-, C-, D-

Se descarta porque fuerza un orden visual artificial y puede interpretarse como una metodología secuencial por fases.

15.4.2 Mantener `engine/`

Se descarta porque puede confundirse con un componente ejecutable y no describe con suficiente precisión el papel de especificaciones, tareas y futuros mecanismos de coordinación.

15.4.3 Utilizar `model/`

Se consideró como representación interna del problema, pero se descarta por su ambigüedad en un contexto de Sistemas Inteligentes, donde `model` se asocia de forma natural a modelos de IA.

15.4.4 Utilizar `deliverables/`

Se descarta a favor de `releases/`, que expresa mejor que los artefactos contenidos en esa área han superado el proceso de aceptación y están preparados para salir del flujo de trabajo.

15.5 Consecuencias

La estructura principal del repositorio pasa a ser legible sin depender de prefijos de orden.

El flujo puede entenderse conceptualmente como:

```
inputs
  ↓
orchestration
  ↓
outputs
  ↓
releases
```

sin que este esquema implique una ejecución estrictamente lineal.

Las herramientas, agentes y personas podrán volver sobre cualquiera de estas áreas cuando el proceso lo requiera.

La distinción entre outputs/ y releases/ hace explícito que **producir un artefacto no implica automáticamente aceptarlo.**

15.6 Principios derivados

Los nombres expresan función, no posición.

El orden visual no debe convertirse en una falsa secuencia metodológica.

Un output es producido; un release ha sido aceptado para salir del proceso.

Part II

Funcional

Esta sección reúne los Engineering Decision Records clasificados en el área **Funcional** (10).

ID	Título
EDR-10001	Usar iasi-home como puerta de entrada visual al ecosistema
EDR-10002	Distinguir libros de productos y artefactos en iasi-home
EDR-10003	Ofrecer iasi-home en castellano e inglés con detección automática
EDR-10004	Generar un catálogo por área de EDR
EDR-10005	Crear EDR mediante un Addin de RStudio
EDR-10006	Crear una presentación de introducción a IASI
EDR-10007	Proporcionar un agente IASI capaz de ejecutar trabajo sobre el workspace

16 Usar iasi-home como puerta de entrada visual al ecosistema

16.1 Contexto

IASI ya no es un único libro con repositorios auxiliares, sino un ecosistema con varios libros, productos y artefactos. Hace falta un punto de entrada que permita orientarse sin conocer previamente su estructura.

16.2 Decisión

`iasi-home` será la portada navegable del ecosistema IASI y concentrará el acceso visual a sus principales piezas.

16.3 Alternativas consideradas

- Usar GitHub como única página de entrada.
- Hacer que cada libro funcione como portada del ecosistema.

16.4 Consecuencias

La navegación comienza por el mapa del ecosistema y después desciende a cada pieza. `iasi-home` no necesita absorber el contenido de los productos, solo presentarlos y enlazarlos.

17 Distinguir libros de productos y artefactos en iasi-home

17.1 Contexto

Los libros y los repositorios reutilizables cumplen funciones diferentes. Presentarlos como una lista homogénea oculta esa diferencia y dificulta entender qué se lee y qué se usa.

17.2 Decisión

iasi-home distinguirá explícitamente los libros de los productos, artefactos o repositorios reutilizables. La estructura visual debe mostrar ambas familias sin confundir publicación editorial con herramienta.

17.3 Alternativas consideradas

- Una única lista de repositorios.
- Clasificar todo como documentación.

17.4 Consecuencias

El visitante entiende antes la naturaleza de cada pieza y puede elegir si quiere leer, explorar una metodología o utilizar un producto.

18 Ofrecer iasi-home en castellano e inglés con detección automática

18.1 Contexto

La landing necesita versiones en castellano e inglés sin duplicar toda la arquitectura del sitio ni obligar al visitante a escoger idioma antes de entrar.

18.2 Decisión

La raíz mantendrá `index_es.qmd` e `index_en.qmd`. `index.qmd` actuará como entrada de detección y el JavaScript común resolverá el idioma al cargar, permitiendo además el cambio explícito entre versiones. Los recursos compartidos permanecerán fuera de árboles específicos de idioma.

18.3 Alternativas consideradas

- Mantener una jerarquía completa `pages/en/`.
- Publicar solo un idioma.
- Duplicar recursos y páginas comunes por idioma.

18.4 Consecuencias

La estructura bilingüe permanece pequeña y predecible. Las páginas comunes pueden tener variantes de idioma sin replicar todo el sitio.

19 Generar un catálogo por área de EDR

19.1 Contexto

Cada área de EDR necesita una entrada navegable incluso cuando todavía no contiene decisiones. Mantener manualmente listas de documentos introduce desajustes en cuanto se crea, mueve o renombra un registro.

19.2 Decisión

Cada directorio de área tendrá un `index.qmd` generado automáticamente. El documento incluirá siempre una introducción que identifique el área y, cuando existan registros, una tabla simple ID | Título con enlaces. Si el área está vacía, mostrará un mensaje explícito en lugar de quedar sin contenido.

19.3 Alternativas consideradas

- Mantener el catálogo a mano.
- No crear páginas de área hasta que exista el primer EDR.
- Mostrar metadatos completos en el catálogo.

19.4 Consecuencias

Todas las áreas son navegables desde el primer día y el catálogo refleja siempre los documentos realmente existentes.

20 Crear EDR mediante un Addin de RStudio

20.1 Contexto

Crear un EDR a mano obliga a recordar área, secuencia, nombre de archivo, ruta y front matter. Es trabajo mecánico y además puede romper la consistencia del registro.

20.2 Decisión

La creación ordinaria de EDR se realizará mediante un Addin de RStudio apoyado en una plantilla. El usuario selecciona el área y proporciona el título; el sistema calcula el siguiente identificador, resuelve el directorio, genera el front matter y abre el documento creado.

20.3 Alternativas consideradas

- Crear archivos manualmente.
- Pedir al usuario que escriba el número de área y la secuencia.
- Mantener un contador externo separado de los documentos.

20.4 Consecuencias

El humano decide el contenido y la clasificación; el sistema mantiene la mecánica de identidad y ubicación. El flujo queda preparado para regenerar el índice de área después de crear el documento.

21 Crear una presentación de introducción a IASI

21.1 Contexto

IASI dispone de libros, documentación, repositorios, artefactos y una presencia web, pero esos medios no sustituyen una presentación concebida específicamente para **introducir IASI de forma progresiva y visual**.

La necesidad no es crear nueva documentación ni resumir un libro. Se necesita un artefacto que permita presentar IASI ante otras personas, explicar sus ideas fundamentales de forma secuencial y apoyar una exposición oral.

La presentación debe poder evolucionar a medida que evolucione la comprensión de IASI y debe formar parte del propio ecosistema de artefactos del proyecto.

21.2 Decisión

Se creará una presentación específica denominada **Introducing IASI**, desarrollada como el proyecto `introducing-iasi`.

Su función será introducir IASI progresivamente, priorizando la comunicación visual y la exposición de conceptos antes que la acumulación de texto explicativo.

La presentación será un artefacto propio de IASI y se desarrollará aplicando el propio enfoque IASI a su construcción.

21.3 Alternativas consideradas

- Utilizar directamente los libros de IASI como material de presentación.
- Utilizar `iasi-home` como sustituto de una presentación.
- Preparar un documento estático de introducción.
- No crear por ahora un artefacto específico de presentación.

Estas alternativas permiten explicar o documentar IASI, pero no proporcionan el tipo de narrativa secuencial, visual y orientada a exposición que requiere una presentación.

21.4 Consecuencias

IASI dispone de un artefacto específico para su presentación pública.

La narrativa de introducción puede evolucionar independientemente de los libros, la documentación y el portal web.

`introducing-iasi` se convierte además en un caso real sobre el que aplicar y validar IASI durante la construcción de un nuevo artefacto.

La tecnología utilizada para materializar la presentación se decide de forma independiente en EDR-20013.

22 Proporcionar un agente IASI capaz de ejecutar trabajo sobre el workspace

22.1 Contexto

La infraestructura local de IASI ya permite ejecutar modelos mediante Ollama.

Sin embargo, disponer de un modelo local no equivale a disponer de un agente capaz de realizar trabajo de ingeniería.

Un modelo utilizado en modo chat o prompt puede recibir contexto y generar una respuesta, pero no dispone necesariamente de las capacidades necesarias para trabajar sobre un proyecto real:

```
prompt
↓
model
↓
response
```

Para ejecutar trabajo IASI se necesita una capa adicional capaz de operar sobre el workspace.

La necesidad se hizo explícita al intentar utilizar los modelos locales como alternativa de ejecución: el modelo estaba disponible, pero faltaba el componente capaz de leer el proyecto, comprender una tarea, utilizar herramientas, modificar los artefactos permitidos y materializar resultados.

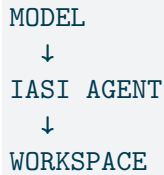
La existencia de agentes externos como Codex demostró además que la tarea y el ejecutor pueden separarse. Una misma tarea IASI puede ser ejecutada por distintos agentes sin redefinir el trabajo.

22.2 Decisión

IASI dispondrá de un agente denominado inicialmente `ias-agent`, capaz de ejecutar trabajo directamente sobre un workspace IASI.

Su objetivo es proporcionar la capa de ejecución que convierte un modelo en un ejecutor de ingeniería.

Conceptualmente:



`iasi-agent` deberá poder, según las reglas y permisos aplicables:

- comprender la tarea solicitada;
- descubrir e inspeccionar la estructura del proyecto;
- leer los artefactos relevantes;
- utilizar herramientas;
- ejecutar comandos cuando corresponda;
- crear o modificar únicamente los artefactos permitidos;
- materializar resultados;
- validar el trabajo realizado.

La capacidad de acceder al workspace no implica autorización para modificar cualquier contenido.

Las reglas de IASI siguen gobernando qué puede leerse, modificarse o producirse. Los artefactos definidos como inmutables continúan siendo inmutables independientemente del agente que ejecute el trabajo.

La ejecución local será una capacidad de primer nivel. `iasi-agent` deberá poder utilizar modelos locales cuando estos sean suficientes para la tarea.

22.3 Alternativas consideradas

22.3.1 Utilizar únicamente Ollama u Open WebUI

Permiten ejecutar e interactuar con modelos locales, pero no proporcionan por sí mismos el agente de workspace que necesita IASI.

Resuelven la inferencia y la conversación, no la ejecución completa del trabajo de ingeniería.

22.3.2 Depender exclusivamente de agentes externos

Agentes como Codex ya ofrecen excelentes capacidades sobre el workspace.

Sin embargo, hacer depender toda la ejecución de IASI de un único agente externo introduciría una dependencia innecesaria entre la metodología y un ejecutor concreto.

Además, disponibilidad, cuotas, conectividad o evolución del producto podrían impedir temporalmente la ejecución sin que haya cambiado la tarea.

22.3.3 Construir inmediatamente un agente completo desde cero

Proporcionaría control total, pero obligaría a desarrollar antes de tiempo capacidades ya resueltas por plataformas agentic existentes: gestión de modelos, herramientas, sesiones, permisos, edición de archivos, ejecución de comandos e integración con protocolos externos.

IASI necesita definir primero qué espera de un agente y reutilizar infraestructura existente siempre que satisfaga esa necesidad.

22.4 Consecuencias

El modelo deja de confundirse con el agente.

Ollama permanece como runtime de modelos y `iasi-agent` proporciona la capa de ejecución sobre el workspace.

IASI puede utilizar diferentes modelos sin redefinir el agente y diferentes agentes sin redefinir la tarea.

Las tareas permanecen independientes del ejecutor.

`iasi-agent` se convierte en uno de los posibles ejecutores de IASI, no en el workflow de IASI.

La arquitectura permite trabajar localmente cuando exista un modelo suficiente para la tarea y utilizar otros ejecutores cuando resulte más apropiado.

Se establece como principio:

El agente es un ejecutor, no el workflow.

Part III

Arquitectura

Esta sección reúne los Engineering Decision Records clasificados en el área **Arquitectura** (20).

ID	Título
EDR-20001	Mantener iasi-edr como repositorio independiente
EDR-20002	Publicar iasi-edr como Quarto book estructurado
EDR-20003	Definir las áreas EDR en _iasi.yml
EDR-20004	Organizar físicamente los EDR por área
EDR-20005	Regenerar los índices EDR en pre-render
EDR-20006	Mantener la lógica específica de EDR fuera de iasi.quarto
EDR-20007	Separar la configuración común de los perfiles HTML y PDF
EDR-20008	Usar la navbar como navegación principal de iasi-edr en HTML
EDR-20009	Componer el estilo HTML con SCSS común y específico
EDR-20010	Mantener iasi-graphics deliberadamente simple
EDR-20011	Crear un espacio dedicado para capacidades agentic de IASI
EDR-20012	Separar el entorno de laboratorios mediante una máquina virtual dedicada
EDR-20013	Usar HTMLSlides para materializar Introducing IASI
EDR-20014	Mantener iasi-agent como proyecto separado de IASI
EDR-20015	Usar OpenCode como base inicial de iasi-agent

23 Mantener iasi-edr como repositorio independiente

23.1 Contexto

Las decisiones afectan a múltiples libros y productos. Si viven dentro de uno de ellos, ese repositorio adquiere una autoridad que no corresponde a decisiones transversales.

23.2 Decisión

iasi-edr será un repositorio independiente del Book, Journey y demás productos de IASI.

23.3 Alternativas consideradas

- Integrar los EDR en el repositorio principal de un libro.
- Mantener un directorio ADR dentro de cada producto.

23.4 Consecuencias

El registro de decisiones tiene ciclo de vida y publicación propios. Cualquier repositorio puede referenciarlo sin dependencia jerárquica.

24 Publicar iasi-edr como Quarto book estructurado

24.1 Contexto

Los EDR son documentos individuales organizados por áreas, pero también forman una publicación consultable como conjunto. Una web plana perdería la relación natural entre área y documentos.

24.2 Decisión

`iasi-edr` será un proyecto Quarto de tipo book y utilizará la estrategia de publicación `structured` de `iasi.quarto`.

24.3 Alternativas consideradas

- Publicarlo como website.
- Mantener solo archivos QMD sin publicación conjunta.
- Construir manualmente el `book.chapters`.

24.4 Consecuencias

Cada área puede representarse como una parte del libro y cada EDR como documento de esa estructura, mientras `iasi.quarto` puede derivar el `_book-structure.yml`.

25 Definir las áreas EDR en `_iasi.yml`

25.1 Contexto

El generador de índices, el futuro Addin y cualquier validación necesitan conocer la correspondencia entre código, nombre y directorio de cada área. Duplicar esa información en scripts y páginas produciría divergencia.

25.2 Decisión

`_iasi.yml` será la fuente estructural legible por máquina para el catálogo de áreas EDR, incluyendo como mínimo `id`, `name` y `path`. `index.qmd` permanece como contenido humano y no se convierte en un contenedor de configuración oculto.

25.3 Alternativas consideradas

- Codificar las áreas directamente en R.
- Mantener la tabla únicamente en `index.qmd`.
- Crear un catálogo duplicado por cada consumidor.

25.4 Consecuencias

Los consumidores comparten una única definición de estructura y el contenido editorial sigue separado de la configuración.

26 Organizar físicamente los EDR por área

26.1 Contexto

La clasificación del identificador debe reflejarse también en la organización física para que el repositorio sea comprensible sin necesidad de ejecutar herramientas.

26.2 Decisión

Los documentos vivirán bajo `edr/<nn-area>/`. La estructura estable incluye los directorios 00-general, 10-functional, 20-architecture, 30-constraints, 40-data, 50-integration, 60-security, 70-quality, 80-operation y 99-pending.

26.3 Alternativas consideradas

- Guardar todos los EDR en un único directorio.
- Separarlos por producto afectado.

26.4 Consecuencias

La ruta física coincide con la clasificación primaria. Un documento tiene una sola ubicación principal, mientras que sus relaciones transversales se expresan mediante metadatos.

27 Regenerar los índices EDR en pre-render

27.1 Contexto

Los índices de área son contenido derivado y deben estar actualizados cuando se publica, sin exigir una acción manual adicional después de crear o modificar un EDR.

27.2 Decisión

Quarto ejecutará `R/create-index.R` mediante `project.pre-render`. El script leerá las áreas desde `_iasi.yml`, recorrerá los EDR de cada directorio, extraerá `id` y `title`, validará la correspondencia con el área y regenerará el `index.qmd` correspondiente.

27.3 Alternativas consideradas

- Actualizar índices desde el Addin únicamente.
- Regenerarlos mediante una tarea manual separada.
- Integrar la lógica directamente en `iasi.quarto`.

27.4 Consecuencias

El render siempre parte de índices frescos. La publicación falla si la estructura física contradice la configuración, evitando resultados silenciosamente incompletos.

28 Mantener la lógica específica de EDR fuera de `iasi.quarto`

28.1 Contexto

`iasi.quarto` proporciona infraestructura genérica de publicación. La generación de catálogos por áreas EDR responde a un modelo particular que, por ahora, no ha demostrado ser necesario en otras publicaciones.

28.2 Decisión

La lógica específica de EDR permanecerá en `iasi-edr`, inicialmente bajo `R/`. Solo se promoverá a `iasi.quarto` cuando exista evidencia de reutilización real.

28.3 Alternativas consideradas

- Incorporar inmediatamente cualquier automatización nueva a `iasi.quarto`.
- Duplicar la lógica entre ambos repositorios.

28.4 Consecuencias

`iasi.quarto` conserva una responsabilidad genérica y EDR puede evolucionar con rapidez sin convertir una necesidad local en contrato global.

29 Separar la configuración común de los perfiles HTML y PDF

29.1 Contexto

HTML y PDF comparten estructura documental pero requieren presentación, recursos y directorios de salida distintos. Mezclar ambas configuraciones en un único YAML hace difícil distinguir qué es común y qué depende del formato.

29.2 Decisión

`_quarto.yml` contendrá la configuración común y declarará el grupo de perfiles. `_quarto-html.yml` y `_quarto-pdf.yml` contendrán respectivamente la configuración específica de HTML y PDF.

29.3 Alternativas consideradas

- Un único `_quarto.yml` con condicionales o configuración acumulada.
- Repositorios separados para cada formato.

29.4 Consecuencias

Cada perfil puede evolucionar sin contaminar al otro y la configuración común permanece visible en un único lugar.

30 Usar la navbar como navegación principal de iasi-edr en HTML

30.1 Contexto

Un Quarto Book ofrece por defecto sidebar de libro y TOC de página. En EDR esos dos elementos duplican la navegación porque las áreas ya forman un catálogo pequeño y estable accesible desde la cabecera.

30.2 Decisión

En el perfil HTML de *iasi-edr* la navegación principal será la navbar con Inicio y las áreas EDR. La sidebar izquierda se desactiva y el TOC flotante derecho también se desactiva.

30.3 Alternativas consideradas

- Mantener la navegación estándar de Quarto Book.
- Usar solo sidebar sin navbar.

30.4 Consecuencias

El contenido gana anchura y el usuario dispone de un único mapa de navegación. Esta decisión afecta solo a HTML y no se incorpora a la configuración común del proyecto.

31 Componer el estilo HTML con SCSS común y específico

31.1 Contexto

El ecosistema necesita identidad visual compartida, pero cada producto puede requerir reglas propias. Un solo fichero global haría que ajustes locales terminasen afectando a publicaciones no relacionadas.

31.2 Decisión

Los estilos vivirán en `resources/css/`. `iasi.scss` contendrá la base visual común y `edr.scss` las especializaciones de EDR. El theme HTML cargará primero el estilo común y después el específico.

31.3 Alternativas consideradas

- Mantener CSS independiente y duplicado en cada producto.
- Incluir todas las reglas EDR en el fichero común.

31.4 Consecuencias

EDR puede sobrescribir detalles sin romper la identidad general. El orden de carga deja explícita la relación base-especialización.

32 Mantener iasi-graphics deliberadamente simple

32.1 Contexto

La mayoría de las necesidades repetibles de IASI son diagramas editoriales sencillos: cajas, círculos, relaciones y composiciones previsibles. Los casos visuales complejos son mucho menos frecuentes y necesitan iteración específica.

32.2 Decisión

`iasi-graphics` se limitará en su fase actual a generar gráficos sencillos, claros y correctos. Las figuras complejas se resolverán excepcionalmente con herramientas de generación de imagen, PowerPoint u otra herramienta adecuada al caso.

32.3 Alternativas consideradas

- Construir desde el inicio un motor gráfico general.
- Resolver manualmente incluso los patrones sencillos repetidos.

32.4 Consecuencias

La herramienta permanece pequeña y automatiza justo la parte repetible. Los casos raros no dictan la arquitectura del caso común.

33 Crear un espacio dedicado para capacidades agentic de IASI

33.1 Contexto

A medida que IASI incorpora comandos, skills, agentes y adaptadores, esas capacidades dejan de ser simples notas de uso de una herramienta concreta y forman una capa reutilizable del ecosistema.

33.2 Decisión

IASI mantendrá un espacio dedicado **agentics** para organizar comandos, skills, adaptadores y demás capacidades agentic que surjan de la metodología, separándolas de los libros y de la implementación de productos concretos.

33.3 Alternativas consideradas

- Guardar cada skill dentro del repositorio donde se utilizó por primera vez.
- Acoplar las capacidades a una única plataforma de agentes.

33.4 Consecuencias

La metodología puede definir capacidades propias y adaptar su ejecución a plataformas como Codex sin convertir la plataforma en la definición del comportamiento.

34 Separar el entorno de laboratorios mediante una máquina virtual dedicada

34.1 Contexto

Los primeros laboratorios pueden realizarse prácticamente desde cualquier entorno, pero a medida que avanzan empiezan a necesitar herramientas, runtimes, servicios, contenedores y productos propios de IASI.

Utilizar para ellos la misma máquina empleada para desarrollar IASI mezclaría dos contextos diferentes. El entorno de desarrollo contiene repositorios fuente, herramientas internas y configuraciones que un usuario de los labs no debería necesitar.

Además, utilizar un entorno separado permite comprobar que los productos IASI funcionan fuera de la máquina en la que fueron desarrollados.

34.2 Decisión

Los laboratorios que requieran una infraestructura específica utilizarán una **máquina virtual dedicada a labs**, separada del entorno `iasi-dev`.

El Lab 10 del Volumen II será responsable de construir ese entorno de referencia. Los laboratorios posteriores podrán asumir la infraestructura creada a partir de ese punto.

La máquina podrá conservar estados reproducibles mediante checkpoints cuando resulte útil para la ejecución o repetición de los laboratorios.

34.3 Alternativas consideradas

- Utilizar directamente la máquina de desarrollo `iasi-dev`.
- Instalar las dependencias de los labs sobre la máquina habitual del usuario.
- Crear un entorno completamente automatizado antes de realizar manualmente los laboratorios de infraestructura.

34.4 Consecuencias

El entorno donde se construye IASI queda separado del entorno donde se utiliza IASI.

Los laboratorios permiten comprobar que los productos pueden instalarse y utilizarse sin depender de sus repositorios fuente.

La máquina virtual proporciona aislamiento y permite conservar estados conocidos para repetir experimentos o recuperar el entorno.

La infraestructura de labs pasa a ser un elemento explícito que deberá mantenerse y evolucionar junto con los propios laboratorios.

35 Usar HTMLSlides para materializar Introducing IASI

35.1 Contexto

Una vez identificada la necesidad de crear una presentación de IASI apareció inmediatamente la cuestión de qué herramienta utilizar para materializarla.

La primera reacción fue asumir **PowerPoint** y buscar la mejor forma de producir la presentación con esa tecnología. A partir de ahí se consideraron también otras herramientas de presentación.

Ese enfoque reveló un problema metodológico: se estaba seleccionando una solución antes de terminar de definir la necesidad.

Esto contradecía un principio fundamental de IASI:

No partir de una herramienta o metodología y preguntar cómo resolver el problema con ella. Partir de la necesidad y evaluar después qué solución la satisface mejor.

Se volvió por tanto a la necesidad.

La presentación debía:

- seguir siendo una **presentación**, no convertirse en un sitio web ni en documentación;
- permitir una narrativa progresiva y controlada;
- ser extremadamente limpia y visual;
- permitir interacción y revelado progresivo cuando resulte útil;
- ofrecer control completo sobre el lenguaje visual de IASI;
- poder mantenerse como un artefacto textual, versionable y reproducible;
- poder ser leído y modificado por agentes dentro del workspace;
- evitar que la identidad visual quede determinada por la herramienta utilizada;
- poder ejecutarse y distribuirse con la menor dependencia posible de un entorno de autoría específico.

Con estos criterios se reevaluaron las alternativas.

35.2 Decisión

La presentación **Introducing IASI** se materializará inicialmente mediante **HTMLSlides**.

HTMLSlides se utilizará como infraestructura de presentación y renderer, no como definición del lenguaje visual de IASI.

El diseño, la tipografía, la composición, los diagramas y el resto de decisiones visuales pertenecen a IASI y deberán poder expresarse posteriormente mediante otros renderers cuando sea necesario.

La salida principal será una presentación HTML ejecutable en navegador y mantenida como artefacto versionable dentro del proyecto.

35.3 Alternativas consideradas

35.3.1 PowerPoint

Ventajas

- Herramienta ampliamente conocida y utilizada.
- Muy buen soporte para edición manual de slides.
- Flujo de presentación maduro.
- Amplias capacidades gráficas, de animación y composición.
- Buena interoperabilidad en contextos corporativos.

Inconvenientes

- El formato de autoría no es especialmente adecuado para inspección y edición textual.
- El control de versiones y la revisión de cambios son menos naturales que sobre fuentes de texto.
- La automatización y modificación por agentes introduce una capa adicional de herramientas.
- Favorece un flujo de trabajo centrado en la herramienta de edición.
- La decisión inicial de usar PowerPoint se había tomado antes de definir correctamente la necesidad.

Resultado

Descartado como tecnología principal para esta presentación. Puede seguir siendo útil para materializaciones o casos concretos futuros.

35.3.2 Gamma

Ventajas

- Generación rápida de presentaciones con asistencia de IA.
- Buenos resultados visuales iniciales con poco trabajo.
- Publicación y compartición web sencillas.
- Reduce considerablemente el esfuerzo de composición inicial.

Inconvenientes

- Introduce dependencia de un servicio externo y de su flujo de trabajo.
- Parte de la lógica de autoría y presentación queda controlada por la plataforma.
- Menor control sobre una fuente canónica completamente gobernada por IASI.
- La reproducibilidad y evolución mediante el workspace son menos directas.
- Sus decisiones visuales y de composición pueden condicionar el lenguaje visual de IASI.

Resultado

Descartado porque optimiza la generación de presentaciones, pero proporciona menos control sobre el artefacto y su proceso de ingeniería.

35.3.3 Prezi

Ventajas

- Narrativa visual diferenciada.
- Permite relaciones espaciales y navegación no lineal.
- Puede producir presentaciones visualmente muy expresivas.

Inconvenientes

- Su modelo de navegación condiciona fuertemente la narrativa.
- El movimiento y la estructura espacial no son una necesidad de Introducing IASI.
- Introduce dependencia de una plataforma y un formato de autoría específicos.
- Resulta menos natural como artefacto textual y versionable dentro del workspace.

Resultado

Descartado porque su principal ventaja no responde a una necesidad identificada y añadiría una forma narrativa impuesta por la herramienta.

35.3.4 HTML desarrollado específicamente para la presentación

Ventajas

- Control completo sobre presentación, comportamiento y lenguaje visual.
- Fuente textual y directamente versionable.
- Ejecución universal mediante navegador.
- Fácil integración con agentes y herramientas del workspace.
- Posibilidad de generar un único artefacto HTML autocontenido.

Inconvenientes

- Obliga a construir y mantener desde cero comportamiento propio de una presentación.
- Navegación, ajuste al viewport, teclado, notas, progresión y otras capacidades básicas pasarían a ser responsabilidad del proyecto.
- Introduciría infraestructura propia antes de demostrar que realmente es necesaria.

Resultado

La aproximación HTML se considera adecuada, pero se evita construir desde cero la infraestructura que ya puede proporcionar una solución especializada.

35.3.5 HTMLSlides

Ventajas

- Mantiene HTML como fuente y como materialización natural.
- Permite trabajar con archivos textuales, versionables e inspeccionables.
- Puede ser leído y modificado directamente por agentes.
- Proporciona una estructura específica de slides y capacidades básicas de presentación.
- Se ejecuta directamente en navegador.
- Permite producir una presentación autocontenida con muy pocas dependencias.
- Conserva un alto grado de control sobre HTML, CSS y comportamiento.
- No obliga a adoptar una identidad visual concreta.
- Permite que el lenguaje visual de IASI se defina independientemente del renderer.

Inconvenientes

- Tiene un ecosistema y una madurez menores que PowerPoint.
- Requiere trabajo propio de diseño y CSS.
- Algunas capacidades avanzadas deberán implementarse o adaptarse cuando aparezca la necesidad.
- La interoperabilidad directa con formatos tradicionales de presentación no es su objetivo principal.

Resultado

Seleccionado porque proporciona suficiente infraestructura de presentación sin apropiarse del diseño ni del proceso de ingeniería.

35.4 Consecuencias

`introducing-iasi` tendrá como materialización principal una presentación HTML basada en HTMLSlides.

El contenido, la narrativa y el lenguaje visual permanecen bajo control de IASI.

HTMLSlides se considera un renderer sustituible. La presentación no debe diseñarse conceptualmente alrededor de las limitaciones o convenciones propias de HTMLSlides.

La elección permite mantener la presentación como artefacto textual, reproducible, versionable y accesible a agentes dentro del workspace.

La decisión podrá revisarse si las necesidades reales de la presentación dejan de estar bien cubiertas por HTMLSlides. La herramienta es consecuencia de la necesidad, no una restricción impuesta al problema.

36 Mantener iasi-agent como proyecto separado de IASI

36.1 Contexto

IASI ya dispone de un runtime y de comandos destinados a soportar la aplicación de la metodología.

Por otra parte, `iasi-dev` es el entorno utilizado para desarrollar IASI.

La aparición de la necesidad de experimentar con un agente local capaz de trabajar sobre el workspace no implica que esa implementación deba incorporarse directamente al runtime de IASI.

Todavía se están descubriendo las capacidades que debe tener un agente IASI, la forma de conectarlo con modelos locales, la relación con skills, comandos y herramientas, y el grado de adaptación necesario sobre plataformas agentic existentes.

Introducir esa experimentación directamente dentro de IASI mezclaría una capacidad todavía en evolución con el núcleo que define y ejecuta la metodología.

36.2 Decisión

Se mantendrá `iasi-agent` como un proyecto separado de IASI.

`iasi-agent` será el espacio donde experimentar, integrar y validar una implementación de agente capaz de ejecutar trabajo IASI sobre un workspace.

La separación conceptual será:

```
IASI
↓
define el trabajo, reglas y workflow

iasi-agent
↓
ejecuta trabajo IASI como uno de sus posibles agentes
```


IASI no dependerá de `ias-agent` para definir la metodología.

`ias-agent` dependerá de los contratos y artefactos que IASI establezca para poder ejecutar trabajo correctamente.

El nombre `ias-agent` identifica esta línea de trabajo y puede evolucionar si la comprensión posterior del producto lo requiere.

36.3 Alternativas consideradas

36.3.1 Incorporar el agente directamente dentro de IASI

Reduciría inicialmente el número de proyectos, pero acoplaría el runtime de la metodología con una implementación agentic concreta todavía experimental.

También dificultaría distinguir qué comportamiento pertenece a IASI y qué comportamiento pertenece al ejecutor.

36.3.2 Tratar el agente como parte de `ias-dev`

`ias-dev` es un entorno de desarrollo, no un producto de ejecución IASI.

Ubicar allí el agente confundiría infraestructura de desarrollo con una capacidad reutilizable del ecosistema.

36.3.3 Utilizar únicamente adaptadores para agentes externos

Los adaptadores siguen siendo necesarios para proyectar capacidades IASI sobre otras plataformas.

Sin embargo, no sustituyen la necesidad de disponer de un espacio propio donde experimentar con un agente IASI y con ejecución local.

36.4 Consecuencias

IASI conserva un núcleo independiente de cualquier implementación agentic concreta.

`ias-agent` puede evolucionar rápidamente, cambiar de plataforma base o incluso descartarse sin obligar a rediseñar el runtime de IASI.

La experimentación local queda aislada del núcleo metodológico.

La decisión se relaciona con EDR-20011, que ya estableció que las capacidades agentic de IASI no deben quedar acopladas a una única plataforma.

La separación permite comprobar en la práctica qué necesita realmente un agente IASI antes de convertir esas necesidades en contratos o capacidades canónicas del sistema.

37 Usar OpenCode como base inicial de iasi-agent

37.1 Contexto

Una vez definida la necesidad de disponer de un agente capaz de trabajar directamente sobre un workspace IASI, la siguiente cuestión fue determinar si debía construirse desde cero o apoyarse sobre una plataforma agentic existente.

La selección no debía comenzar por la herramienta.

Primero se identificaron las capacidades necesarias:

- trabajar desde el workspace real;
- leer y modificar archivos de forma controlada;
- ejecutar comandos;
- utilizar modelos locales servidos por Ollama;
- no quedar ligado a un único proveedor de modelos;
- permitir reglas e instrucciones de proyecto;
- incorporar herramientas externas;
- soportar MCP;
- permitir capacidades reutilizables como skills y comandos;
- ofrecer control sobre permisos;
- poder utilizarse desde terminal;
- permitir posteriormente integración programática;
- mantener separadas la metodología IASI y la plataforma de ejecución.

A partir de estas necesidades se evaluaron varias alternativas.

37.2 Decisión

`iasi-agent` utilizará inicialmente **OpenCode** como plataforma base de ejecución agentic.

OpenCode se considera infraestructura sustituible.

IASI seguirá siendo propietario de las tareas, reglas, skills, contratos y workflow que definan el comportamiento esperado. OpenCode proporciona las capacidades de ejecución necesarias para que un modelo pueda operar sobre el workspace.

Conceptualmente:

```
IASI
↓
tasks / rules / skills / workflow
↓
adapter / configuration
↓
OpenCode
↓
model provider
↓
Ollama / cloud / other
```

La elección de OpenCode no convierte sus convenciones en el modelo conceptual de IASI.

Cuando una capacidad IASI necesite proyectarse al formato esperado por OpenCode, esa traducción deberá mantenerse como una adaptación de plataforma.

37.3 Alternativas consideradas

37.3.1 Ollama + Open WebUI

Ventajas

- Infraestructura local ya disponible.
- Sencillez de instalación y operación.
- Independencia de proveedores cloud para la inferencia.
- Adecuado para conversación, análisis y uso interactivo de modelos.

Inconvenientes

- Ollama es un runtime de modelos, no un agente de workspace.
- Open WebUI proporciona principalmente una interfaz de interacción con los modelos.
- No resuelve por sí mismo la ejecución controlada de tareas sobre archivos, terminal y herramientas del proyecto.

Resultado

Se mantiene como infraestructura de modelos e interfaz de conversación, pero no como base suficiente de `ias-agent`.

37.3.2 Continue

Ventajas

- Dispone de modo Agent con lectura, edición de archivos y ejecución de comandos.
- Puede utilizar Ollama y modelos locales.
- Ofrece herramientas integradas y soporte para MCP.
- Integra bien el contexto del código y del workspace desde el IDE.
- Permite reglas y políticas de herramientas.

Inconvenientes

- Su experiencia principal está integrada en IDEs.
- Vincular `ias-agent` a Continue haría que la capacidad agentic quedase demasiado asociada a una extensión y a un entorno de desarrollo concreto.
- IASI necesita que el agente pueda existir independientemente del editor utilizado.

Resultado

Alternativa válida para utilizar IASI desde un IDE, pero no seleccionada como base principal de `ias-agent`.

37.3.3 Aider

Ventajas

- Herramienta madura orientada a terminal.
- Excelente integración con repositorios Git.
- Puede trabajar con modelos locales mediante Ollama.
- Dispone de repo map para proporcionar contexto del repositorio.
- Ofrece un flujo sólido de edición, diff, commit y undo.
- Puede editar también documentación y archivos de configuración, no únicamente código.

Inconvenientes

- Su modelo está especialmente orientado a pair programming y edición de repositorios Git.
- El flujo de trabajo gira alrededor de conversaciones de edición, selección de archivos y commits.

- Encaja peor como plataforma general sobre la que proyectar las distintas capacidades agentic de IASI.
- La extensión mediante skills, agentes, permisos y herramientas no constituye el centro de su modelo del mismo modo que en OpenCode.

Resultado

Muy buena alternativa para edición asistida de código, pero menos adecuada como sustrato general de un agente IASI.

37.3.4 Codex CLI

Ventajas

- Agente de terminal maduro capaz de leer, modificar y ejecutar trabajo sobre el workspace.
- Ya ha sido utilizado en proyectos IASI.
- Dispone de controles de aprobación y sandbox.
- Puede utilizar modelos OpenAI y también modelos locales mediante Ollama.
- Demuestra en la práctica que una tarea IASI puede ser ejecutada por un agente independiente.

Inconvenientes

- Es una plataforma agentic definida y evolucionada dentro del ecosistema OpenAI.
- Utilizarla como identidad de `iasi-agent` acercaría la implementación propia de IASI a una plataforma concreta.
- El uso mediante servicios y modelos OpenAI puede estar sujeto a disponibilidad y cuotas.
- IASI quiere conservar la posibilidad de experimentar sobre una base deliberadamente neutral respecto al proveedor.

Resultado

Se mantiene como ejecutor válido de tareas IASI y como referencia funcional, pero no se adopta como identidad ni base principal de `iasi-agent`.

37.3.5 Construir un agente desde cero

Ventajas

- Control completo de arquitectura, comportamiento y contratos.
- Ninguna dependencia conceptual de una plataforma agentic existente.
- Posibilidad de implementar exclusivamente las capacidades necesarias para IASI.

Inconvenientes

- Obliga a reconstruir capacidades ya disponibles: sesiones, tool calling, edición, shell, permisos, modelos, MCP, interfaces y gestión del contexto.
- Aumenta drásticamente el trabajo antes de haber descubierto qué partes necesitan realmente una implementación propia.
- Violenta el criterio de no construir una solución nueva cuando una infraestructura existente satisface la necesidad.

Resultado

Descartado como punto de partida. Podrán sustituirse componentes concretos cuando exista una razón de ingeniería para hacerlo.

37.3.6 OpenCode

Ventajas

- Es un agente de código abierto y orientado al workspace.
- Puede utilizar múltiples proveedores y modelos locales mediante Ollama.
- Dispone de herramientas para operar sobre el código base y permite extenderlas con herramientas propias.
- Soporta servidores MCP locales y remotos.
- Permite definir y controlar permisos de herramientas.
- Soporta agentes configurables.
- Soporta instrucciones de proyecto.
- Permite comandos personalizados definidos en Markdown.
- Soporta skills mediante archivos `SKILL.md`.
- Puede utilizarse desde terminal sin depender de un IDE.
- Ofrece ejecución headless mediante servidor HTTP.
- Dispone de SDK para integración programática.
- Su arquitectura permite separar cliente, servidor, modelo y workspace.

Inconvenientes

- Sigue siendo una dependencia externa que puede evolucionar.
- Sus convenciones (`AGENTS.md`, `.opencode/`, configuración, etc.) no coinciden necesariamente con las abstracciones canónicas de IASI.
- Los modelos locales pequeños pueden limitar la calidad del tool calling y del razonamiento agentic.
- Será necesario mantener una capa de adaptación para evitar que IASI dependa de formatos específicos de OpenCode.

Resultado

Seleccionado como base inicial porque proporciona la mayor parte de la infraestructura necesaria sin obligar a IASI a construir un agente desde cero ni a ligarse a un proveedor de modelos o a un IDE concreto.

37.4 Consecuencias

`iasi-agent` podrá comenzar como una adaptación/configuración de OpenCode en lugar de como un agente implementado completamente desde cero.

Los modelos servidos por Ollama podrán utilizarse como proveedores de inferencia para el agente.

Las reglas, skills y comandos de IASI no deberán definirse canónicamente en formatos propios de OpenCode. Se proyectarán mediante adapters cuando sea necesario.

La integración con OpenCode deberá respetar las reglas de permisos e inmutabilidad establecidas por IASI.

OpenCode se considera sustituible.

Si en el futuro deja de satisfacer las necesidades de IASI, podrá reemplazarse sin cambiar la definición de las tareas ni el workflow.

La decisión sigue el mismo criterio aplicado en otras partes del ecosistema:

Utilizar una herramienta para lo que resuelve bien sin convertirla en la definición del problema.

Part IV

Constraints

Esta sección reúne los Engineering Decision Records clasificados en el área **Constraints** (30).

ID	Título
EDR-30001	Diseñar las publicaciones IASI para escritorio
EDR-30002	Evitar la generalización prematura
EDR-30003	Construir manualmente la infraestructura antes de automatizarla
EDR-30004	Usar Windows como sistema operativo de referencia para los labs

38 Diseñar las publicaciones IASI para escritorio

38.1 Contexto

El uso principal previsto para las publicaciones IASI es escritorio. Ajustar continuamente layouts, tamaños y componentes para móvil añade trabajo y condiciona una interfaz que se está diseñando y evaluando en pantallas de escritorio.

38.2 Decisión

La referencia de diseño será escritorio. No se añadirán media queries ni ajustes responsive específicos para móvil salvo que una necesidad futura los justifique expresamente.

38.3 Alternativas consideradas

- Diseñar mobile-first.
- Mantener paridad visual obligatoria entre escritorio y móvil desde el inicio.

38.4 Consecuencias

El CSS permanece más pequeño y la revisión visual se hace contra una referencia estable. La ausencia de trabajo móvil es deliberada, no un olvido.

39 Evitar la generalización prematura

39.1 Contexto

IASI genera patrones que podrían parecer reutilizables desde su primera aparición. Convertir inmediatamente cada patrón en infraestructura compartida crea APIs y dependencias antes de conocer sus verdaderos límites.

39.2 Decisión

Una solución permanecerá local mientras solo exista una necesidad demostrada. La promoción a componente común se hará cuando aparezca reutilización real y pueda definirse un contrato estable.

39.3 Alternativas consideradas

- Centralizar desde el primer uso.
- Diseñar extensibilidad para escenarios todavía hipotéticos.

39.4 Consecuencias

La arquitectura común crece por evidencia. Los productos conservan libertad para experimentar sin comprometer prematuramente a todo el ecosistema.

40 Construir manualmente la infraestructura antes de automatizarla

40.1 Contexto

Automatizar una máquina o entorno que todavía no se comprende convierte supuestos provisionales en código y dificulta distinguir necesidad real de accidente de la primera instalación.

40.2 Decisión

La construcción inicial de la máquina y la infraestructura se hará manualmente. Después de experimentar, aprender, documentar y especificar el entorno, se automatizarán las partes estables.

40.3 Alternativas consideradas

- Empezar directamente con infraestructura totalmente automatizada.
- Mantener permanentemente una instalación manual.

40.4 Consecuencias

La automatización captura conocimiento comprobado en lugar de congelar descubrimientos provisionales. El proceso manual inicial forma parte del aprendizaje.

41 Usar Windows como sistema operativo de referencia para los labs

41.1 Contexto

Los laboratorios necesitan un entorno de referencia concreto para poder describir instalaciones, comandos, rutas, herramientas y procedimientos de manera reproducible.

Intentar documentar desde el principio cada laboratorio simultáneamente para Windows, Linux, macOS y otras combinaciones multiplicaría las variantes y desviaría el contenido del objetivo del propio laboratorio.

IASI no pretende depender de un único sistema operativo, pero los labs necesitan un camino principal que podamos ejecutar, verificar y mantener.

41.2 Decisión

Windows será el sistema operativo de referencia por defecto para los laboratorios de IASI.

Las instrucciones principales de los labs se escribirán y verificarán sobre Windows.

Esto no implica que IASI sea un proyecto exclusivo de Windows ni que los laboratorios no puedan realizarse sobre otros sistemas operativos. Otros entornos podrán utilizarse cuando sea posible, pero no será obligatorio mantener una variante equivalente de cada paso.

41.3 Alternativas consideradas

- Utilizar Linux como sistema operativo de referencia.
- Mantener instrucciones equivalentes para varios sistemas operativos desde el inicio.
- No establecer ningún sistema operativo de referencia.

41.4 Consecuencias

Los laboratorios disponen de una plataforma base conocida sobre la que pueden escribirse instrucciones reproducibles.

Se reduce la complejidad editorial y de validación al evitar una matriz de procedimientos por sistema operativo.

La metodología y los productos IASI deberán continuar evitando dependencias innecesarias de Windows cuando no formen parte de una necesidad real.

La compatibilidad con otros sistemas podrá desarrollarse y validarse posteriormente sin condicionar el diseño inicial de cada lab.

Part V

Datos

Esta sección reúne los Engineering Decision Records clasificados en el área **Datos** (40).

ID	Título
EDR-40001	Usar el front matter como fuente de metadatos del EDR
EDR-40002	Tratar los índices de área como datos derivados
EDR-40003	Hacer los EDR consultables por metadatos y texto completo

42 Usar el front matter como fuente de metadatos del EDR

42.1 Contexto

Los índices, búsquedas, validaciones y herramientas de creación necesitan leer información estructurada de cada decisión sin interpretar su prosa.

42.2 Decisión

Cada EDR almacenará en el front matter, como mínimo, `id`, `title`, `status`, `area`, `created`, `tags`, `affects`, `related`, `supersedes` y `superseded-by`. El cuerpo del documento conserva el razonamiento.

42.3 Alternativas consideradas

- Inferir metadatos del nombre del archivo.
- Mantener un catálogo YAML externo con todos los documentos.
- Extraer clasificación mediante análisis del texto.

42.4 Consecuencias

Cada documento es autocontenido y legible por máquina. Las herramientas pueden indexarlo sin mantener una segunda base de datos de metadatos.

43 Tratar los índices de área como datos derivados

43.1 Contexto

Un `index.qmd` de área repite identificadores y títulos que ya existen en los EDR. Si se edita como fuente primaria puede quedar desincronizado del conjunto real de documentos.

43.2 Decisión

Los EDR y su front matter son la fuente de verdad. Los `index.qmd` de área se regeneran a partir de ellos y pueden reemplazarse completamente en cada ejecución.

43.3 Alternativas consideradas

- Considerar el índice como catálogo maestro.
- Exigir actualización manual simultánea del EDR y del índice.

43.4 Consecuencias

No hay doble escritura. Un índice puede borrarse y reconstruirse sin pérdida de información.

44 Hacer los EDR consultables por metadatos y texto completo

44.1 Contexto

Clasificar cada decisión en una sola área primaria no basta para responder preguntas transversales como qué decisiones afectan a un producto, cuáles están relacionadas o qué decisiones sustituyen a otras.

44.2 Decisión

El modelo EDR incorporará metadatos de afectación y relaciones, y la publicación debe permitir consulta tanto por esos campos como por el texto completo del documento.

44.3 Alternativas consideradas

- Crear múltiples copias del mismo EDR en cada área afectada.
- Limitar la recuperación a la navegación por carpetas.

44.4 Consecuencias

La clasificación física sigue siendo simple mientras las relaciones transversales se expresan como datos. El registro puede evolucionar hacia búsquedas y vistas derivadas sin cambiar los documentos.

Part VI

Integración

Esta sección reúne los Engineering Decision Records clasificados en el área **Integración** (50).

ID	Título
EDR-50001	Distinguir MCP del servidor MCP
EDR-50002	Exponer PlantUML como capacidad utilizable por modelos
EDR-50003	Clasificar los productos técnicos por su forma de uso

45 Distinguir MCP del servidor MCP

45.1 Contexto

Usar “MCP” para referirse tanto al protocolo como al software que se instala y configura mezcla dos conceptos distintos y dificulta explicar qué producto ofrece IASI.

45.2 Decisión

MCP se utilizará para nombrar el Model Context Protocol. El componente ejecutable que expone capacidades a un modelo se denominará `servidor MCP`.

45.3 Alternativas consideradas

- Llamar MCP indistintamente al protocolo y al servidor.
- Inventar un nombre específico de IASI para evitar la terminología estándar.

45.4 Consecuencias

La documentación puede explicar con precisión qué estándar se usa y qué componente debe instalarse, configurarse y operarse.

46 Exponer PlantUML como capacidad utilizable por modelos

46.1 Contexto

Generar diagramas PlantUML desde un modelo requiere una frontera clara entre la intención del agente y la herramienta externa que procesa el diagrama.

46.2 Decisión

PlantUML se tratará como una herramienta que puede exponerse mediante un servidor MCP para que un modelo solicite la creación o renderizado de diagramas sin incorporar PlantUML al propio modelo.

46.3 Alternativas consideradas

- Acoplar la generación a una interfaz manual.
- Tratar PlantUML como parte intrínseca del agente.

46.4 Consecuencias

La capacidad puede reutilizarse desde distintos modelos o plataformas y conserva una interfaz explícita de integración.

47 Clasificar los productos técnicos por su forma de uso

47.1 Contexto

El ecosistema contiene componentes de naturaleza distinta. Llamarlos a todos “herramientas” oculta cómo se consumen y qué debe instalar o configurar el usuario.

47.2 Decisión

La documentación distinguirá al menos tres tipos de producto técnico: **Herramienta**, **Extensión** y **Servidor MCP**. Cada producto se presenta según su forma real de integración.

47.3 Alternativas consideradas

- Usar “MCP” como categoría de producto.
- Presentar todos los repositorios bajo una única etiqueta genérica.

47.4 Consecuencias

La navegación y la documentación pueden comunicar de inmediato el modo de consumo esperado de cada pieza.

Part VII

Seguridad

Esta sección reúne los Engineering Decision Records clasificados en el área **Seguridad (60)**.
Actualmente no hay EDR registrados en esta área.

Part VIII

Calidad

Esta sección reúne los Engineering Decision Records clasificados en el área **Calidad** (70).

ID	Título
EDR-70001	Validar la estructura Quarto antes de preparar o renderizar
EDR-70002	Fallar ante inconsistencias estructurales del registro EDR
EDR-70003	Regenerar de forma determinista los artefactos derivados
EDR-70004	Preservar el formato deliberado del código Go

48 Validar la estructura Quarto antes de preparar o renderizar

48.1 Contexto

Un pipeline puede avanzar bastante antes de fallar si faltan archivos esenciales o la configuración no es coherente con el tipo de proyecto.

48.2 Decisión

`iasi.quarto` validará al inicio la existencia de `_quarto.yml` y `_iasi.yml`, determinará el tipo de proyecto Quarto y verificará la configuración específica requerida por ese tipo antes de continuar con `prepare/build`. Para publicaciones con perfiles se comprobará la presencia de las configuraciones declaradas.

48.3 Alternativas consideradas

- Delegar todos los errores a `quarto render`.
- Validar solo después de haber generado artefactos.

48.4 Consecuencias

Los errores de configuración aparecen cerca de su causa y antes de modificar artefactos derivados. El pipeline puede detenerse con mensajes específicos de IASI.

49 Fallar ante inconsistencias estructurales del registro EDR

49.1 Contexto

Omitir silenciosamente un directorio, un identificador incorrecto o un front matter incompleto permitiría publicar un catálogo aparentemente correcto pero parcial.

49.2 Decisión

Las herramientas EDR fallarán ante inconsistencias relevantes: directorios configurados que no existen, documentos sin `id` o `title`, identificadores que no corresponden al área y duplicados detectables.

49.3 Alternativas consideradas

- Emitir warnings y continuar siempre.
- Ignorar archivos problemáticos durante la generación del índice.

49.4 Consecuencias

Un render fallido es preferible a una publicación incompleta que parezca válida. Los errores estructurales se convierten en defectos visibles y corregibles.

50 Regenerar de forma determinista los artefactos derivados

50.1 Contexto

Los archivos generados introducen incertidumbre si su contenido depende del estado previo o de modificaciones manuales.

50.2 Decisión

Los artefactos derivados se reconstruirán completamente a partir de sus fuentes y con orden estable. En EDR, los índices se ordenarán por identificador y no conservarán contenido manual entre ejecuciones.

50.3 Alternativas consideradas

- Actualizar incrementalmente los índices existentes.
- Preservar secciones manuales dentro de archivos generados.

50.4 Consecuencias

Dos ejecuciones sobre las mismas fuentes producen el mismo resultado. Los cambios generados son revisables y no esconden estado acumulado.

51 Preservar el formato deliberado del código Go

51.1 Contexto

IASI disponía de una rule específica para Go que exigía que todo el código fuente se ajustase al formato producido por `gofmt` y desaconsejaba mantener manualmente cualquier formato que entrase en conflicto con él.

Durante la revisión del código de `iasi-cli` se comprobó que determinadas decisiones de formato pueden expresar de forma directa la estructura lógica del programa y mejorar su lectura por personas.

Por ejemplo, un `switch` utilizado como tabla de decisión puede escribirse de forma que sus alternativas se comprendan de un vistazo:

```
switch adapter.Action {
    case cli.Help: adapter = helpAdapter(adapter)
    case cli.List: adapter = listAdapters(adapter)
    default:      adapter = processAdapter(adapter)
}
```

La indentación muestra que los `case` pertenecen al bloque del `switch`, mientras que la alineación permite comparar visualmente las alternativas y sus acciones.

El formateo automático de Go modifica esta representación aunque el código sea válido, eliminando información visual introducida deliberadamente para facilitar su comprensión.

51.2 Decisión

El código Go propiedad de IASI conservará su **formato deliberado**.

No se ejecutarán automáticamente `gofmt`, `go fmt` ni herramientas equivalentes que normalicen el formato del código Go y puedan sustituir decisiones explícitas de indentación, alineación o agrupación visual.

La rule existente que obliga a utilizar `gofmt` se elimina de `iasi-core`.

Las herramientas de desarrollo, automatizaciones y procesos de integración no deberán aplicar formateo automático sobre código Go propiedad del proyecto.

El formato seguirá siendo responsabilidad de la ingeniería del código. Puede modificarse cuando la modificación sea deliberada y mejore o preserve su legibilidad, pero no se normalizará automáticamente por el mero hecho de ajustarse a una convención externa.

51.3 Rationale

El formato del código no es únicamente decoración. Puede formar parte de su representación y ayudar a mostrar relaciones, niveles, alternativas y bloques conceptuales.

IASI prioriza que el código pueda **leerse y comprenderse por personas**. Una herramienta automática es útil mientras contribuya a ese objetivo, pero no debe imponerse cuando elimina información visual que el autor ha introducido deliberadamente para expresar mejor la estructura del programa.

La decisión no rechaza las convenciones del ecosistema Go como referencia general. Establece que una convención de formato no tiene prioridad automática sobre una representación deliberada que resulte más clara para el código concreto.

51.4 Alternativas consideradas

51.4.1 Mantener `gofmt` como obligatorio

Se descarta porque impide conservar formatos deliberados que permiten representar con mayor claridad determinadas estructuras lógicas.

51.4.2 Utilizar `gofmt` por defecto y permitir excepciones puntuales

Se descarta porque cualquier ejecución automática puede destruir esas excepciones. Obliga además a introducir mecanismos adicionales para identificar qué bloques deben preservarse.

51.4.3 Desactivar únicamente el formateo al guardar

Es insuficiente. El mismo problema puede aparecer mediante `go fmt`, `gofmt`, tareas de build, acciones del editor, agentes o pipelines de integración.

51.5 Consecuencias

El código Go de IASI puede utilizar indentación, alineación y espacios de manera deliberada cuando ayuden a expresar su estructura.

Los editores y agentes que trabajen sobre el código no deben ejecutar formateadores automáticos de Go salvo decisión explícita que modifique este EDR.

Las pipelines de build y validación no deben asumir que `gofmt` forma parte de los criterios de aceptación del código.

El formato pasa a revisarse como parte de la legibilidad del código, no como conformidad automática con la salida de una herramienta.

El código generado o mantenido externamente puede conservar el formato impuesto por su generador o por su fuente original cuando no sea propiedad del proyecto.

51.6 Principios derivados

El formato del código puede expresar estructura.

La legibilidad para las personas tiene prioridad sobre la normalización automática.

El código propiedad de IASI se formatea deliberadamente, no automáticamente.

Part IX

Operación

Esta sección reúne los Engineering Decision Records clasificados en el área **Operación** (80).

ID	Título
EDR-80001	Usar HTML como perfil por defecto
EDR-80002	Separar los directorios de salida por perfil
EDR-80003	Usar scripts R reutilizables para preview y render
EDR-80004	Separar herramientas principales de trabajo y laboratorios locales
EDR-80005	Postprocesar la salida de Quarto sin forzar su comportamiento

52 Usar HTML como perfil por defecto

52.1 Contexto

Durante la edición y revisión ordinaria la salida HTML es la más rápida y navegable. PDF sigue siendo una publicación necesaria, pero no necesita activarse implícitamente en cada render.

52.2 Decisión

Los proyectos Quarto con perfiles HTML/PDF usarán `html` como perfil por defecto. PDF se solicitará de forma explícita mediante su perfil.

52.3 Alternativas consideradas

- No declarar perfil por defecto.
- Renderizar ambos perfiles siempre.
- Usar PDF como perfil predeterminado.

52.4 Consecuencias

El comando de render habitual produce HTML sin opciones adicionales y el coste de PDF se paga solo cuando se necesita esa salida.

53 Separar los directorios de salida por perfil

53.1 Contexto

HTML produce un sitio con múltiples archivos mientras PDF produce un artefacto documental distinto. Mezclar ambas salidas dificulta limpieza, despliegue y revisión.

53.2 Decisión

El perfil HTML escribirá en `outputs/html` y el perfil PDF en `outputs/pdf`. El `output-dir` pertenece a la configuración específica de cada perfil.

53.3 Alternativas consideradas

- Un único directorio de salida para todos los formatos.
- Configurar el directorio común en `_quarto.yml`.

53.4 Consecuencias

Cada formato puede limpiarse, publicarse o inspeccionarse de manera independiente.

54 Usar scripts R reutilizables para preview y render

54.1 Contexto

Repetir comandos largos de Quarto en terminal dispersa conocimiento operativo y facilita variaciones entre ejecuciones.

54.2 Decisión

IASI utilizará scripts R reutilizables para las operaciones recurrentes de preview y render, como mínimo para HTML y PDF, en lugar de depender únicamente de comandos manuales recordados por el usuario.

54.3 Alternativas consideradas

- Ejecutar siempre comandos Quarto manualmente.
- Ocultar todo el proceso dentro del IDE sin scripts versionados.

54.4 Consecuencias

Las operaciones frecuentes quedan documentadas como código y pueden evolucionar junto al proyecto.

55 Separar herramientas principales de trabajo y laboratorios locales

55.1 Contexto

El entorno de trabajo puede aprovechar servicios como Codex y Copilot sin renunciar a experimentar con modelos locales. Obligar a todos los componentes a usar el mismo backend reduciría el valor del laboratorio.

55.2 Decisión

Codex y Copilot se utilizarán como herramientas principales del workspace cuando resulten adecuadas. Ollama se mantendrá para experimentación local y laboratorios, incluyendo el desarrollo exploratorio de *iasi-graphics*.

55.3 Alternativas consideradas

- Estandarizar todo IASI sobre un único proveedor o runtime de modelos.
- Excluir modelos locales del entorno por no ser la herramienta principal.

55.4 Consecuencias

Los experimentos pueden evaluar capacidades locales sin condicionar el flujo principal. La metodología y los productos permanecen desacoplados de una única plataforma de IA.

56 Postprocesar la salida de Quarto sin forzar su comportamiento

56.1 Contexto

Quarto resuelve correctamente su responsabilidad principal: transformar las fuentes del proyecto en salidas HTML, PDF u otros formatos soportados. Sin embargo, algunas necesidades editoriales, estructurales o de publicación de IASI no encajan de forma natural en su modelo de configuración o en su ciclo de renderizado.

Intentar resolver esas necesidades forzando `_quarto.yml`, perfiles, Pandoc, filtros, hooks u otros mecanismos internos aumenta el acoplamiento con Quarto y convierte cada excepción en una negociación con la herramienta.

IASI no necesita que Quarto conozca todas sus reglas. Necesita una salida correcta de Quarto sobre la que pueda aplicar después sus propias transformaciones.

56.2 Decisión

IASI utilizará Quarto dentro de los límites naturales de su responsabilidad y aplicará, cuando sea necesario, una fase explícita de **postprocesado** sobre la salida generada.

El flujo conceptual será:

```
source
↓
Quarto
↓
output de Quarto
↓
postprocesado IASI
↓
validación
↓
artefacto final
```

Quarto realizará el renderizado. IASI actuará después sobre el resultado para resolver necesidades que no corresponda implementar dentro de Quarto.

El postprocesado podrá incluir, entre otras operaciones, ajustes editoriales, transformación o reorganización de recursos, incorporación de metadatos, empaquetado y preparación de artefactos para publicación.

No se modificará ni forzará el comportamiento interno de Quarto cuando la misma necesidad pueda resolverse de forma más simple y desacoplada después del renderizado.

56.3 Alternativas consideradas

- Resolver todas las necesidades mediante configuración nativa de Quarto.
- Introducir filtros, hooks o extensiones para cada ajuste requerido por IASI.
- Modificar la salida fuente antes del render para inducir el resultado deseado.
- Mantener lógica específica de publicación mezclada con la configuración de Quarto.

56.4 Consecuencias

Quarto queda tratado como un motor de renderizado con una responsabilidad clara, mientras IASI conserva el control de su propio proceso de publicación.

Las particularidades de IASI pueden evolucionar sin depender de detalles internos de Quarto ni aumentar innecesariamente la complejidad de su configuración.

La salida directa de Quarto pasa a ser un artefacto intermedio cuando exista postprocesado. El artefacto final es el resultado posterior a las transformaciones y validaciones de IASI.

Esta decisión establece además un criterio general de ingeniería: **no luchar contra una herramienta para obligarla a resolver aquello que queda fuera de su responsabilidad natural; utilizarla para lo que hace bien y continuar el proceso fuera de ella.**

Part X

Pendiente de clasificar

Esta sección reúne los Engineering Decision Records clasificados en el área **Pendiente de clasificar** (99).

Actualmente no hay EDR registrados en esta área.

57 Áreas EDR

58 Áreas EDR

Los Engineering Decision Records de IASI utilizan identificadores con la forma:

```
EDR-nnmmm
```

donde:

- **nn** identifica el **área principal de ingeniería** de la decisión.
- **mmm** identifica secuencialmente la decisión dentro de esa área.

El área permite conocer la naturaleza principal de una decisión antes incluso de abrir el EDR.

58.1 Áreas

O, si quieres la cabecera compacta, solo para echo:

```
::: {.cell tbl-colwidths='[10,20,70]'}  
::: {.cell-output-display}
```

nn	Área	Criterio
:--:	:-----	:-----
00	General	Decisiones transversales o relativas al propio funcionamiento
10	Funcional	Qué debe hacer el sistema y qué comportamiento debe ofrecer.
20	Arquitectura	Estructura del sistema, componentes, responsabilidades y relaciones
30	Constraints	Restricciones que condicionan o limitan las posibles soluciones
40	Datos	Modelos, persistencia, intercambio, ciclo de vida y gobierno de datos
50	Integración	Interfaces, protocolos y comunicación entre sistemas o componentes
60	Seguridad	Identidad, acceso, protección, amenazas y controles.
70	Calidad	Validación, pruebas, observabilidad y atributos de calidad.
80	Operación	Despliegue, ejecución, configuración, infraestructura y explotación
99	Pendiente de clasificar	Decisiones registradas provisionalmente cuya clasificación definitiva

```

:::
:::

## Reglas de clasificación

Cada EDR pertenece a una única área principal.

El área representa la naturaleza de la decisión, no:

- la herramienta utilizada;
- la tecnología elegida;
- el repositorio donde se implementa;
- el equipo responsable.

Cuando una decisión afecta a varios ámbitos, se asigna al área que mejor represente el objeto.

Los efectos secundarios pueden expresarse mediante los metadatos del EDR.

## Área `00` - General

`00` está reservado para decisiones generales o transversales.

También pertenece a esta área cualquier decisión relacionada con el propio sistema de Ingeniería.

Por ejemplo:

``` text
EDR-00001

```

puede definir el modelo, estructura y reglas de los propios EDR.

00 no debe utilizarse simplemente porque una decisión resulte difícil de clasificar.

## 58.2 Área 10 — Funcional

El área 10 agrupa decisiones relacionadas con **qué debe hacer el sistema**.

Incluye decisiones sobre:

- comportamiento funcional;
- capacidades ofrecidas;



- reglas de negocio;
- interacción entre el sistema y sus usuarios;
- resultados esperados;
- criterios funcionales.

Una decisión pertenece a esta área cuando su cuestión principal puede formularse aproximadamente como:

¿Qué comportamiento debe ofrecer el sistema?

## 58.3 Área 20 — Arquitectura

El área 20 agrupa decisiones relacionadas con **cómo se estructura el sistema**.

Incluye decisiones sobre:

- componentes;
- responsabilidades;
- relaciones entre componentes;
- separación de responsabilidades;
- organización interna;
- estilos y patrones arquitectónicos;
- distribución de capacidades;
- fronteras entre subsistemas.

Una decisión pertenece a esta área cuando su cuestión principal puede formularse aproximadamente como:

¿Cómo organizamos el sistema para satisfacer sus necesidades?

## 58.4 Área 30 — Constraints

El área 30 agrupa decisiones derivadas de **restricciones que condicionan el espacio de soluciones posibles**.

Incluye restricciones:

- técnicas;
- organizativas;
- operativas;
- regulatorias;
- económicas;
- temporales;

- de compatibilidad;
- de plataforma;
- impuestas por sistemas externos.

Una decisión pertenece a esta área cuando su cuestión principal parte de una condición que no puede ignorarse libremente.

Los constraints no describen necesariamente qué queremos hacer, sino **qué límites debe respetar cualquier solución válida**.

## 58.5 Área 40 — Datos

El área 40 agrupa decisiones relacionadas con la **representación, persistencia, intercambio y ciclo de vida de los datos**.

Incluye decisiones sobre:

- modelos de datos;
- estructuras y formatos;
- almacenamiento y persistencia;
- migraciones;
- retención y ciclo de vida;
- calidad y gobierno del dato;
- intercambio y transformación de información.

Una decisión pertenece a esta área cuando el elemento principal sobre el que se decide es el dato y su tratamiento.

## 58.6 Área 50 — Integración

El área 50 agrupa decisiones relacionadas con la **comunicación entre sistemas, servicios o componentes**.

Incluye decisiones sobre:

- interfaces;
- APIs;
- protocolos;
- mensajería;
- contratos de integración;
- interoperabilidad;
- mecanismos de comunicación;
- dependencias con sistemas externos.

Una decisión pertenece a esta área cuando su cuestión principal es cómo se relacionan o intercambian información distintos elementos del sistema.

## 58.7 Área 60 — Seguridad

El área 60 agrupa decisiones relacionadas con la **protección del sistema, sus usuarios y su información**.

Incluye decisiones sobre:

- identidad;
- autenticación;
- autorización;
- gestión de secretos;
- protección de datos;
- amenazas;
- controles;
- aislamiento;
- trazabilidad de seguridad.

Una decisión pertenece a esta área cuando su propósito principal es reducir riesgos o establecer mecanismos de protección.

## 58.8 Área 70 — Calidad

El área 70 agrupa decisiones relacionadas con la **validación del sistema y sus atributos de calidad**.

Incluye decisiones sobre:

- pruebas;
- acceptance tests;
- validación;
- verificación;
- observabilidad;
- métricas;
- rendimiento;
- fiabilidad;
- mantenibilidad;
- criterios de calidad.

Una decisión pertenece a esta área cuando define cómo demostrar, medir o asegurar que el sistema cumple las propiedades esperadas.

## 58.9 Área 80 — Operación

El área 80 agrupa decisiones relacionadas con la **ejecución y explotación del sistema**.

Incluye decisiones sobre:

- despliegue;
- configuración;
- infraestructura;
- entornos;
- ejecución;
- automatización operativa;
- monitorización;
- recuperación;
- mantenimiento en producción.

Una decisión pertenece a esta área cuando su cuestión principal es cómo ejecutar, desplegar, configurar o mantener el sistema en funcionamiento.

## 58.10 Área 99 — Pendiente de clasificar

99 está reservado para decisiones que deben registrarse, pero cuya clasificación definitiva todavía no está clara.

Su uso es **provisional**.

Debe utilizarse para evitar dos problemas:

- retrasar el registro de una decisión porque todavía no se sabe dónde clasificarla;
- forzar una clasificación prematura que después resulte incorrecta.

99 no es una categoría residual ni un destino permanente.

Cuando se determine el área correcta, la decisión debe trasladarse a su clasificación definitiva.

## 58.11 Criterio de asignación

Para determinar **nn**, debe identificarse primero la naturaleza principal de la decisión.

Puede utilizarse esta secuencia:

1. ¿Cuál es el problema que estamos resolviendo?
2. ¿Qué aspecto de la ingeniería estamos decidiendo?
3. ¿Dónde tendría sentido buscar esta decisión dentro de varios años?
4. ¿Seguiría perteneciendo a la misma área si cambiase la tecnología utilizada?

La respuesta determina el área principal.

En caso de duda entre varias áreas, debe seleccionarse aquella que mejor represente **la razón por la que existe la decisión**.

Si todavía no existe información suficiente para hacerlo con criterio, se utiliza provisionalmente el área 99.

## 58.12 Una única clasificación principal

Una decisión puede afectar simultáneamente a múltiples ámbitos.

Por ejemplo, una decisión arquitectónica puede tener consecuencias funcionales, operativas o de seguridad.

Esto no implica que deba pertenecer a varias áreas.

Cada EDR tiene un único **nn**.

Los demás efectos se expresan mediante metadatos como:

```
tags:
 - publication
 - validation

affects:
 - iasi-quarto
 - iasi-book-I

related:
 - EDR-20002
```

El identificador proporciona la clasificación principal.

Los metadatos proporcionan el contexto adicional.

## 58.13 El área no depende de la implementación

`nn` no debe asignarse según:

- el lenguaje de programación;
- el framework;
- la herramienta;
- el producto;
- el repositorio;
- el proveedor;
- el agente de IA que implemente la decisión.

Por ejemplo, una decisión sobre cómo se estructura la publicación de IASI puede implementarse inicialmente en `iasi-quarto`.

Eso no convierte automáticamente la decisión en una decisión sobre Quarto.

La pregunta relevante es qué se está decidiendo desde el punto de vista de ingeniería.

## 58.14 Estabilidad

Una vez asignado un código `nn` a un área:

- su significado no cambia;
- no se reutiliza para otro ámbito;
- los EDR clasificados conservan su identificador.

Los códigos de área son permanentes.

El área 99 constituye la excepción: representa explícitamente un estado provisional pendiente de clasificación.

## 58.15 Separación entre áreas

Los códigos se asignan con separación deliberada:

```
00
10
20
30
40
50
```

60  
70  
80  
99

Esto mantiene una clasificación legible y deja espacio para incorporar nuevas áreas si la evolución de IASI lo hace necesario.

## 58.16 Creación de nuevas áreas

La lista de áreas no pretende definir anticipadamente todos los ámbitos posibles de la ingeniería.

Se crearán nuevas áreas cuando las decisiones reales de IASI hagan necesaria una nueva categoría estable.

Antes de crear una nueva área debe comprobarse que:

1. ninguna de las áreas existentes representa adecuadamente la decisión;
2. el nuevo ámbito tiene significado propio desde el punto de vista de ingeniería;
3. no representa simplemente una herramienta, tecnología o producto;
4. previsiblemente podrá contener múltiples decisiones;
5. su significado puede mantenerse estable en el tiempo.

La taxonomía debe evolucionar a partir de necesidades reales.

## 58.17 Principio de evolución

IASI no pretende construir anticipadamente una clasificación completa de todas las decisiones de ingeniería posibles.

Las áreas se incorporarán cuando aparezcan decisiones reales que justifiquen su existencia.

**Las áreas no se inventan por anticipado. Se descubren cuando las decisiones de ingeniería las hacen necesarias.**

## 58.18 Regla fundamental

**nn clasifica la naturaleza principal de la decisión. Los metadatos describen todo lo demás.**

El identificador debe permitir saber dónde buscar una decisión.

El contenido y los metadatos explican su alcance, sus relaciones y sus consecuencias.